

Development of Flow at the Entrance of a Cylindrical Pipe

Numerical Simulation and Modelling of Incompressible Flow

LV-Nr.: 321.053 EX
Winter Semester 2025/26

Felix Scope
`felix_scope@yahoo.de`

September 12, 2026

Contents

Assignment	3
Statement on the Use of AI Tools	6
1 Problem Description	7
1.1 Geometry and Setup	7
1.2 Physical Parameters	7
2 Mathematical Formulation	8
2.1 Governing Equations in Cylindrical Coordinates	8
2.1.1 Continuity Equation	8
2.1.2 Momentum Equations	8
2.1.3 Energy Equation	9
2.2 Boundary Conditions	9
2.3 Non-Dimensionalization	10
2.3.1 Reference Scales	10
2.3.2 Non-Dimensional Variables	10
2.3.3 Non-Dimensional Governing Equations	10
2.3.4 Non-Dimensional Boundary Conditions	10
3 Numerical Solution	11
3.1 Computational Domain and Grid	11
3.2 Spatial Discretization: Finite Volume Method	11
3.2.1 Continuity (at scalar CV)	11
3.2.2 Convective Scheme: 2nd-Order Upwind (Linear Upwind Differencing)	12
3.2.3 Temperature equation and cell-by-cell development	12
3.2.4 Axial Momentum (at w-face CV)	13
3.3 Temporal Discretization	13
3.4 Pressure Correction Method (Projection)	14
3.5 Poisson Solver	15
3.6 Ghost Cells and Boundary Implementation	15
4 Implementation	16
4.1 Code Structure	16
4.2 Algorithm Flowchart	18
4.3 Key Implementation Details	18
5 Analytical Solution	20
5.1 Velocity Profile (Hagen–Poiseuille)	20
5.2 Temperature Profile	20
5.3 Validity of the Fully-Developed Temperature Profile	20
5.4 Nusselt Number	21
6 Results	23
6.1 Simulation Parameters	23
6.2 Convergence History	23
6.3 Velocity Field	24
6.4 Temperature Field	25

6.5	Velocity Profiles at Selected Axial Positions	26
6.6	Temperature Profiles at Selected Axial Positions	27
6.7	Comparison with Analytical Solution	28
7	Conclusion	29
A	2D Incompressible Flow in Cylindrical Coordinates (r, z) — Velocities (u, w)	30
A.1	Governing Equations (Strong Form)	30
A.2	Staggered Grid Layout	31
A.3	Finite Volume Discretisation — Integration over $\Delta r \Delta z$	31
A.3.1	Continuity Equation	31
A.3.2	Axial Momentum: CV at $(i, j + \frac{1}{2})$	31
A.3.3	Radial Momentum: CV at $(i + \frac{1}{2}, j)$	32
A.4	Projection Method	32
A.4.1	Step 1 — Predictor	32
A.4.2	Step 2 — Poisson Equation for Pressure Correction p'	33
A.4.3	Step 3 — Velocity Correction	33
A.4.4	Step 4 — Pressure Update	33
A.5	Poisson Solver Procedure	33
A.6	Ghost Cells and Boundary Conditions	34
B	MATLAB Source Code	35
B.1	Configuration and Initialisation	35
B.2	Main Simulation Script	35
B.3	Grid Generation	40
B.4	Boundary Conditions	40
B.5	Momentum Solver	42
B.6	Poisson Solver and Pressure Correction	47
B.7	Temperature Solver	51
B.8	Convergence Monitoring	53
B.9	Analytical Reference Solutions	54
B.10	Plotting Helper	54

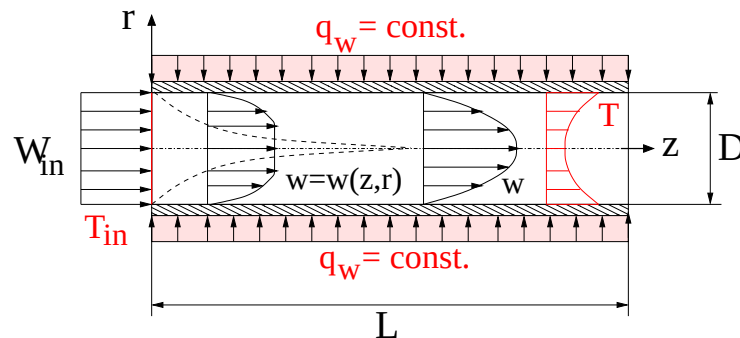
Assignment

Numerical Simulation and Modelling of Incompressible Flow

LV-Nr.: 321.053 **EX**, WS 2025/26

Entwicklung der Strömung am Eintritt eines zylindrischen Rohrs

Es soll eine zwei-dimensionale laminare Strömung in einem zylindrischen Rohr mit gegebenem Durchmesser D simuliert werden. Die gesamte Rohrwand wird mit konstanter Wärmestromdichte $q_w = \text{const.}$ beheizt. Das inkompressible Fluid habe gegebene konstante Dichte $\rho = \text{const.}$, dynamische Viskosität $\mu = \text{const.}$, Wärmeleitfähigkeit $\lambda = \text{const.}$, und spezifische Wärmekapazität $c_v = \text{const.}$. Viskose Dissipation kann in der thermischen Energiebilanz vernachlässigt werden. Die axiale Länge des Rohrs sei $L = 50D$. Die Strömung tritt mit einem gleichförmigen Geschwindigkeits- und Temperaturprofil $w|_{z=0} = W_{in} = \text{const.}$ bzw. $T|_{z=0} = T_{in} = \text{const.}$ in das Rohr ein. Das Strömungsgebiet ist unten stehend schematisch skizziert.



Aufgaben:

- Formulieren Sie die Erhaltungsgleichungen, welche dieses zwei-dimensionale Strömungsproblem beschreiben, in Zylinderkoordinaten.
- Entdimensionalisieren Sie die Bewegungsgleichungen unter Verwendung von D , W_{in} , ρW_{in}^2 , und $q_w / \rho c_v W_{in}$ als Bezugsgrößen für Länge, Geschwindigkeit, Druck, und Temperaturdifferenz $T - T_{in}$.
- Diskretisieren Sie die dimensionslosen Gleichungen
 - räumlich mittels Finite-Volumen Methode zweiter Ordnung genau
 - zeitlich mit einem expliziten Verfahren zweiter Ordnung genau.
- Programmieren Sie ein geeignetes direktes Lösungsverfahren zur Lösung der Poisson Gleichung, wie im Rahmen der Druckkorrektur zur Erfüllung der Kontinuitätsgleichung erforderlich.
- Simulieren Sie die Strömung für den Fall mit $Re = \rho W_{in} D / \mu = 100$, $Pr = \mu c_v / \lambda = 5$ bis das Strömungsfeld einen konvergierten Zustand erreicht hat.
- Dokumentieren Sie Ihre Arbeit. Bei der Darstellung der Ergebnisse sollen insbesondere gezeigt werden:
 - Kontouren der dimensionslosen Geschwindigkeitsfelder in beide Richtungen, also von w/W_{in} (axial) und u/W_{in} (radial), sowie Temperaturfelder $(T(x, y) - T_{in}) \rho W_{in} c_v / q_w$.
 - Profile von $w(r, z) / W_{in}$ und $(T(x, y) - T_{in}) \rho W_{in} c_v / q_w$ an ausgewählten Stromab-Positionen $z_i = \text{const.}$

Korrigierte Seite 2. Seite 1 ist unverändert. Auf dieser Seite ist *nur* die voll entwickelte Vergleichs-Temperaturformel berichtigt: in der Originalfassung fehlte der Faktor $\frac{1}{2}$ vor $Re Pr$ in den radialen Termen, und die additive Konstante muss $-\frac{7}{48}$ statt $-\frac{3}{4}$ lauten, damit die Mischtemperatur $\theta_{\text{bulk}}(z) = 4z/D$ für alle $z \geq 0$ korrekt ist. Die unveränderte Originalfassung liegt als **Assignment_WS25_error.pdf** bei; Herleitung und Literaturquellen ($Nu = 48/11$) sind im begleitenden Bericht angegeben.

- Fügen Sie zum Vergleich die Profile für die voll entwickelte zylindrische Rohrströmung hinzu, hier gegeben als

$$w = 2 W_{\text{in}} \left[1 - \left(\frac{2r}{D} \right)^2 \right],$$

$$\frac{(T - T_{\text{in}}) \varrho W_{\text{in}} c_v}{q_w} = 4 \frac{z}{D} + Re Pr \left[\frac{1}{2} \left(\frac{2r}{D} \right)^2 - \frac{1}{8} \left(\frac{2r}{D} \right)^4 - \frac{7}{48} \right],$$

welche für zunehmende Lauflänge z schlussendlich erreicht werden müssen.

Äquivalente Schreibweise (Faktor ausgeklammert):

$$\frac{(T - T_{\text{in}}) \varrho W_{\text{in}} c_v}{q_w} = 4 \frac{z}{D} + \frac{1}{2} Re Pr \left[\left(\frac{2r}{D} \right)^2 - \frac{1}{4} \left(\frac{2r}{D} \right)^4 - \frac{7}{24} \right].$$

Statement on the Use of AI Tools

AI-based tools (an LLM coding assistant, Claude) were used while preparing both the MATLAB solver and this report: for drafting and debugging portions of the MATLAB code, for typesetting and organising sections of the report and several of its schematic figures, and for wording and proofreading assistance.

Every governing equation, discretisation choice, boundary condition, and numerical result presented here was worked through, checked, and is understood by the author. The AI tool was used as an implementation and editing aid – to speed up typing, catch mistakes, and keep the document consistent – not as a substitute for the underlying engineering judgement: the derivation of the equations, the finite-volume discretisation and projection method, the choice of numerical schemes, and the interpretation of the results were all reasoned through and can be explained and defended independently of the tool, as required at the individual oral assessment.

1 Problem Description

1.1 Geometry and Setup

The problem considers two-dimensional laminar flow in a cylindrical pipe with diameter D and length $L = 50D$. The pipe wall is heated with constant heat flux $q_w = \text{const.}$. The incompressible fluid has constant properties: density ρ , dynamic viscosity μ , thermal conductivity λ , and specific heat capacity c_v . Viscous dissipation is neglected.

The flow enters with uniform velocity and temperature profiles:

$$w|_{z=0} = W_{\text{in}} = \text{const.} \quad (1)$$

$$T|_{z=0} = T_{\text{in}} = \text{const.} \quad (2)$$

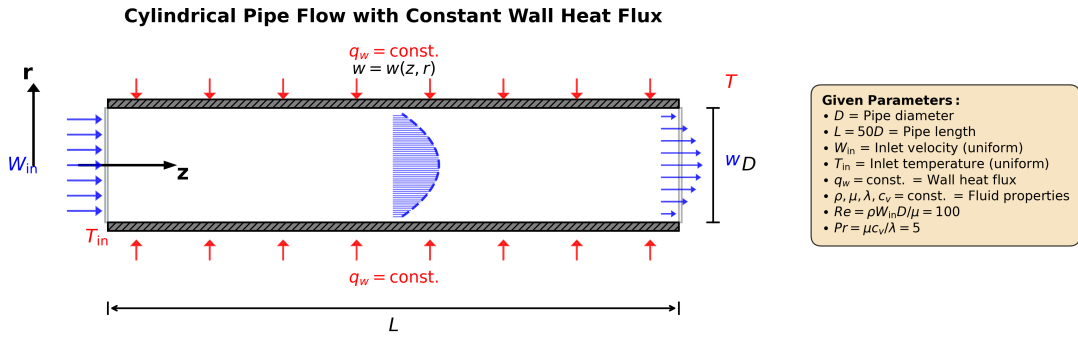


Figure 1: Schematic of the cylindrical pipe flow with constant wall heat flux. Flow enters with uniform velocity W_{in} and temperature T_{in} , and develops into a parabolic profile along the pipe length.

1.2 Physical Parameters

- Reynolds number: $Re = \frac{\rho W_{\text{in}} D}{\mu} = 100$
- Prandtl number: $Pr = \frac{\mu c_v}{\lambda} = 5$
- Length-to-diameter ratio: $L/D = 50$

2 Mathematical Formulation

2.1 Governing Equations in Cylindrical Coordinates

For axisymmetric flow in cylindrical coordinates (r, z) with velocity components (u, w) in the radial and axial directions, the governing equations in **conservation form** are:

2.1.1 Continuity Equation

$$\underbrace{\frac{1}{r} \frac{\partial(r u)}{\partial r}}_{\text{radial mass-flux divergence}} + \underbrace{\frac{\partial w}{\partial z}}_{\text{axial mass-flux divergence}} = 0 \quad (3)$$

The $1/r$ weighting on the radial term accounts for the circumferential area $2\pi r$ growing with radius; the two terms summing to zero enforces incompressibility (no net accumulation of mass in any control volume).

2.1.2 Momentum Equations

Radial momentum:

$$\frac{\partial u}{\partial t} + \frac{1}{r} \frac{\partial(r u^2)}{\partial r} + \frac{\partial(u w)}{\partial z} = -\frac{1}{\rho} \frac{\partial p}{\partial r} + \frac{1}{\rho} \left[\frac{1}{r} \frac{\partial(r \tau_{rr})}{\partial r} - \frac{\tau_{\varphi\varphi}}{r} + \frac{\partial \tau_{rz}}{\partial z} \right] \quad (4)$$

Axial momentum:

$$\frac{\partial w}{\partial t} + \frac{1}{r} \frac{\partial(r u w)}{\partial r} + \frac{\partial(w^2)}{\partial z} = -\frac{1}{\rho} \frac{\partial p}{\partial z} + \frac{1}{\rho} \left[\frac{1}{r} \frac{\partial(r \tau_{rz})}{\partial r} + \frac{\partial \tau_{zz}}{\partial z} \right] \quad (5)$$

Term by term, each momentum equation reads (local rate of change) + (convection) = (pressure force) + (viscous force):

- $\frac{\partial u}{\partial t}, \frac{\partial w}{\partial t}$ — **local (unsteady) acceleration**: the rate of change of momentum stored in a fixed control volume.
- $\frac{1}{r} \frac{\partial(r u^2)}{\partial r}, \frac{\partial(u w)}{\partial z}$ (and the analogous axial terms) — **convective transport**: momentum carried in and out of the control volume by the flow itself (advection by u and w).
- $-\frac{1}{\rho} \frac{\partial p}{\partial r}, -\frac{1}{\rho} \frac{\partial p}{\partial z}$ — **pressure-gradient force** per unit mass, the driving force pushing fluid from high to low pressure.
- $\frac{1}{\rho} \left[\frac{1}{r} \frac{\partial(r \tau_{rr})}{\partial r} + \frac{\partial \tau_{rz}}{\partial z} \right]$ and the axial counterpart — **viscous (diffusive) force** per unit mass, the net effect of internal friction spreading momentum from fast- to slow-moving fluid.
- $-\frac{\tau_{\varphi\varphi}}{\rho r}$ — a **curvature (“hoop-stress”) term** unique to cylindrical coordinates: it arises because the circumferential stress $\tau_{\varphi\varphi}$ acts on a surface whose area changes with r ; there is no Cartesian analogue.

The stress tensor components for a Newtonian fluid are:

$$\underbrace{\tau_{rr} = 2\mu \frac{\partial u}{\partial r}}_{\text{radial normal stress}}, \quad \underbrace{\tau_{\varphi\varphi} = 2\mu \frac{u}{r}}_{\text{hoop (circumferential) normal stress}}, \quad \underbrace{\tau_{zz} = 2\mu \frac{\partial w}{\partial z}}_{\text{axial normal stress}}, \quad \underbrace{\tau_{rz} = \mu \left(\frac{\partial u}{\partial z} + \frac{\partial w}{\partial r} \right)}_{\text{shear stress}} \quad (6)$$

2.1.3 Energy Equation

$$\underbrace{\frac{\partial T}{\partial t}}_{\text{local rate of change}} + \underbrace{\frac{1}{r} \frac{\partial(r u T)}{\partial r} + \frac{\partial(w T)}{\partial z}}_{\text{convective transport of heat by the flow}} = \underbrace{\frac{\lambda}{\rho c_v} \left[\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial T}{\partial r} \right) + \frac{\partial^2 T}{\partial z^2} \right]}_{\text{conductive (diffusive) transport, Fourier's law}} \quad (7)$$

Viscous dissipation (heat generated by internal friction) is neglected, per the problem statement.

2.2 Boundary Conditions

Inlet ($z = 0$):

$$u = 0, \quad w = W_{\text{in}}, \quad T = T_{\text{in}} \quad (8)$$

Wall ($r = D/2$):

$$u = 0, \quad w = 0, \quad \lambda \frac{\partial T}{\partial r} \bigg|_{r=D/2} = q_w \quad (9)$$

Heat flows into the fluid at the wall (constant heating), so with r increasing outward this is a positive gradient: the wall is hotter than the centreline.

Centreline ($r = 0$, symmetry):

$$u = 0, \quad \frac{\partial w}{\partial r} \bigg|_{r=0} = 0, \quad \frac{\partial T}{\partial r} \bigg|_{r=0} = 0 \quad (10)$$

Outlet ($z = L$, zero-gradient):

$$\frac{\partial u}{\partial z} = 0, \quad \frac{\partial w}{\partial z} = 0, \quad \frac{\partial T}{\partial z} = 0 \quad (11)$$

2.3 Non-Dimensionalization

2.3.1 Reference Scales

- Length: D
- Velocity: W_{in}
- Pressure: ρW_{in}^2
- Temperature difference: $\frac{q_w}{\rho c_v W_{\text{in}}}$
- Time: D/W_{in}

2.3.2 Non-Dimensional Variables

$$r^* = \frac{r}{D}, \quad z^* = \frac{z}{D}, \quad t^* = \frac{t W_{\text{in}}}{D} \quad (12)$$

$$u^* = \frac{u}{W_{\text{in}}}, \quad w^* = \frac{w}{W_{\text{in}}}, \quad p^* = \frac{p}{\rho W_{\text{in}}^2} \quad (13)$$

$$\theta = \frac{(T - T_{\text{in}}) \rho c_v W_{\text{in}}}{q_w} \quad (14)$$

2.3.3 Non-Dimensional Governing Equations

Dropping asterisks for brevity (all quantities are non-dimensional hereafter):

Continuity:

$$\frac{1}{r} \frac{\partial(r u)}{\partial r} + \frac{\partial w}{\partial z} = 0 \quad (15)$$

Radial momentum:

$$\frac{\partial u}{\partial t} + \frac{1}{r} \frac{\partial(r u^2)}{\partial r} + \frac{\partial(u w)}{\partial z} = -\frac{\partial p}{\partial r} + \frac{1}{\text{Re}} \left[\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial u}{\partial r} \right) - \frac{u}{r^2} + \frac{\partial^2 u}{\partial z^2} \right] \quad (16)$$

Axial momentum:

$$\frac{\partial w}{\partial t} + \frac{1}{r} \frac{\partial(r u w)}{\partial r} + \frac{\partial(w^2)}{\partial z} = -\frac{\partial p}{\partial z} + \frac{1}{\text{Re}} \left[\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial w}{\partial r} \right) + \frac{\partial^2 w}{\partial z^2} \right] \quad (17)$$

Energy:

$$\frac{\partial \theta}{\partial t} + \frac{1}{r} \frac{\partial(r u \theta)}{\partial r} + \frac{\partial(w \theta)}{\partial z} = \frac{1}{\text{RePr}} \left[\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial \theta}{\partial r} \right) + \frac{\partial^2 \theta}{\partial z^2} \right] \quad (18)$$

2.3.4 Non-Dimensional Boundary Conditions

Inlet ($z = 0$): $u = 0, w = 1, \theta = 0$

Wall ($r = 1/2$): $u = 0, w = 0, \partial \theta / \partial r = \text{RePr}$

Non-dimensionalizing the wall condition directly: $\partial \theta / \partial r^* = D \cdot (\rho c_v W_{\text{in}} / q_w) \cdot \partial T / \partial r|_{\text{wall}} = D \cdot (\rho c_v W_{\text{in}} / q_w) \cdot (q_w / \lambda) = \rho c_v W_{\text{in}} D / \lambda = \text{RePr}$ (i.e. 500 for $\text{Re} = 100, \text{Pr} = 5$), *not* ± 1 — the reference length/temperature scales do not cancel.

Centreline ($r = 0$): $u = 0, \partial w / \partial r = 0, \partial \theta / \partial r = 0$

Outlet ($z = 50$): $\partial u / \partial z = 0, \partial w / \partial z = 0, \partial \theta / \partial z = 0$

3 Numerical Solution

3.1 Computational Domain and Grid

The computational domain is:

$$\Omega = \{(r, z) : 0 \leq r \leq 0.5, \quad 0 \leq z \leq 50\} \quad (19)$$

A **staggered grid** (MAC grid) is used:

- Scalar variables (p, θ) : cell centres $(r_{c,i}, z_{c,j})$
- Radial velocity u : radial faces $(r_{f,i}, z_{c,j})$
- Axial velocity w : axial faces $(r_{c,i}, z_{f,j})$

Grid parameters: $n_r = 50$ cells radially, $n_z = 250$ cells axially, giving:

$$\Delta r = \frac{0.5}{n_r} = 0.01, \quad \Delta z = \frac{50}{n_z} = 0.2 \quad (20)$$

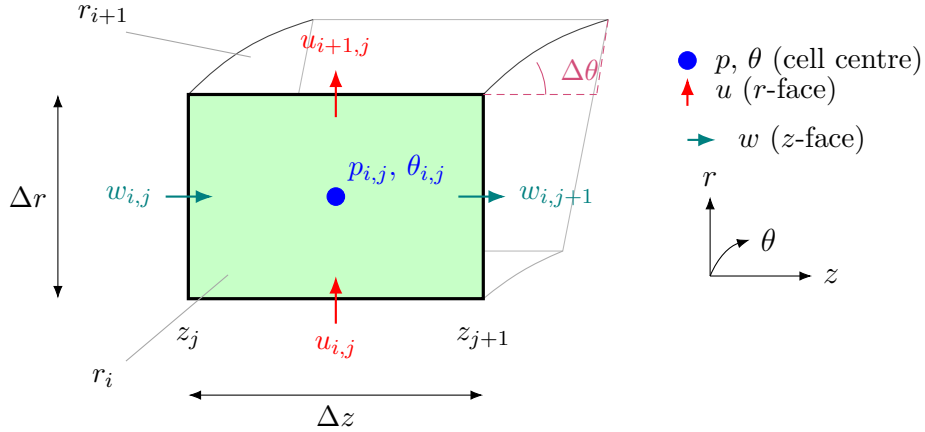


Figure 2: Staggered grid in the (r, z) plane. Scalars (p, θ) at the cell centre $(r_{c,i}, z_{c,j})$ (blue); radial velocity u on the (curved, cylindrical) r -faces (red), stored at $u_{i,j}$ and $u_{i+1,j}$; axial velocity w on the z -faces (teal), stored at $w_{i,j}$ and $w_{i,j+1}$. The control volume is drawn swept through a finite $\Delta\theta$ purely to convey the 3-D cylindrical shape of the r -faces (note the outer face at r_{i+1} sweeps further than the inner face at r_i , since arc length $= r \Delta\theta$); the axisymmetric problem itself has $\partial/\partial\theta = 0$ and no θ -dependence.

3.2 Spatial Discretization: Finite Volume Method

The governing equations are integrated over each control volume using the FVM. After applying Gauss' divergence theorem, the volume integrals become surface flux integrals that are evaluated at control-volume faces.

3.2.1 Continuity (at scalar CV)

$$\frac{r_{i+\frac{1}{2}} u_{i+1,j} - r_{i-\frac{1}{2}} u_{i,j}}{r_{c,i} \Delta r} + \frac{w_{i,j+1} - w_{i,j}}{\Delta z} = 0 \quad (21)$$

3.2.2 Convective Scheme: 2nd-Order Upwind (Linear Upwind Differencing)

Central differencing requires the cell Peclet number $Pe = |v| \Delta x / (\nu_{\text{eff}})$ to satisfy $Pe < 2$. For our grid:

$$Pe_z = |w| \Delta z Re = 1 \times 0.2 \times 100 = 20 \gg 2 \quad (22)$$

(and $Pe_\theta = |w| \Delta z RePr = 100$ for the energy equation, even higher).

Pure central differencing is therefore prone to non-physical oscillations at these cell Peclet numbers. We use **2nd-order upwind (linear-upwind / LUD) differencing** for convective terms: at each face, the transported quantity is linearly extrapolated from the two cells on the *upwind* side of the face, based on the sign of the transporting velocity. On a uniform grid and for a smooth solution, this face reconstruction has a truncation error of $\mathcal{O}(\Delta x^2)$, so the convective discretization is formally second-order accurate, as required by the assignment. This is an accuracy statement, not a boundedness guarantee: because the extrapolated face value is not limited, LUD can overshoot or undershoot near sharp transients, boundaries, or insufficiently resolved gradients. Thus the scheme is second-order in smooth regions but is not monotone or TVD everywhere.

For a generic flux $F = v \phi$ at the face between cells i and $i + 1$:

$$\phi_{\text{face}} = \begin{cases} \frac{3}{2}\phi_i - \frac{1}{2}\phi_{i-1} & \text{if } v_{\text{face}} > 0 \\ \frac{3}{2}\phi_{i+1} - \frac{1}{2}\phi_{i+2} & \text{if } v_{\text{face}} < 0 \end{cases} \quad (23)$$

which reduces to the classic 1st-order upwind value (ϕ_i or ϕ_{i+1}) if the linear extrapolation term is dropped.

Diffusive terms retain standard **second-order central differencing**.

Why these reconstructions are second order. Expand ϕ in a Taylor series about the face location x_f (face between cells i and $i+1$, spacing Δx , upwind case $v_{\text{face}} > 0$): with $x_i = x_f - \frac{1}{2}\Delta x$ and $x_{i-1} = x_f - \frac{3}{2}\Delta x$,

$$\phi_i = \phi_f - \frac{1}{2}\Delta x \phi'_f + \frac{1}{8}\Delta x^2 \phi''_f + \mathcal{O}(\Delta x^3), \quad \phi_{i-1} = \phi_f - \frac{3}{2}\Delta x \phi'_f + \frac{9}{8}\Delta x^2 \phi''_f + \mathcal{O}(\Delta x^3). \quad (24)$$

Plain 1st-order upwind ($\phi_{\text{face}} = \phi_i$) retains the $-\frac{1}{2}\Delta x \phi'_f$ term and is therefore only $\mathcal{O}(\Delta x)$ accurate. Forming the linear-upwind combination $\frac{3}{2}\phi_i - \frac{1}{2}\phi_{i-1}$ cancels the ϕ'_f term *exactly* ($-\frac{3}{2} \cdot \frac{1}{2}\Delta x + \frac{1}{2} \cdot \frac{3}{2}\Delta x = 0$), leaving $\phi_{\text{face}} = \phi_f - \frac{3}{8}\Delta x^2 \phi''_f + \mathcal{O}(\Delta x^3)$: the leading error is $\mathcal{O}(\Delta x^2)$, confirming LUD is second order. The central-difference diffusion operator is second order by the same mechanism applied symmetrically: expanding $\phi_{i\pm 1}$ about x_i and summing, the odd-order ($\Delta x, \Delta x^3$) terms cancel by symmetry, leaving $(\phi_{i+1} - 2\phi_i + \phi_{i-1})/\Delta x^2 = \phi''_i + \frac{1}{12}\Delta x^2 \phi''''_i + \mathcal{O}(\Delta x^4)$.

3.2.3 Temperature equation and cell-by-cell development

The temperature variable $\theta_{i,j}$ is stored at the centre of every scalar control volume. At each time step, its value is advanced from the net transport and diffusion through the four faces of that control volume:

$$\frac{d\theta_{i,j}}{dt} = -\frac{1}{r_{c,i}\Delta r} [(ru\theta)_e - (ru\theta)_w] - \frac{(w\theta)_n - (w\theta)_s}{\Delta z} + \frac{1}{RePr} \left[\frac{r_{i+\frac{1}{2}}\theta_r|_e - r_{i-\frac{1}{2}}\theta_r|_w}{r_{c,i}\Delta r} + \frac{\theta_z|_n - \theta_z|_s}{\Delta z} \right]. \quad (25)$$

The convective face temperatures are reconstructed with the LUD formula above, while the diffusive face gradients use central differences. The wall face receives the prescribed heat flux through the ghost value $\theta_g = \theta_{i,j} + \Delta r \text{ RePr}$; the inlet face uses $\theta = 0$ and the outlet uses a zero axial gradient. Thus every cell is updated from its local radial and axial neighbour values, together with the local face velocities. In the AB2 time integrator the resulting cellwise right-hand side is stored and updated as

$$\theta_{i,j}^{n+1} = \theta_{i,j}^n + \Delta t \left(\frac{3}{2} R_{\theta,i,j}^n - \frac{1}{2} R_{\theta,i,j}^{n-1} \right), \quad (26)$$

with a forward-Euler startup step when the previous right-hand side is unavailable. Heat therefore first enters cells adjacent to the wall and then spreads radially and downstream; the centreline and downstream cells develop later because they receive heat through diffusion and axial advection rather than directly from the heated wall.

3.2.4 Axial Momentum (at w-face CV)

The w-CV at $(r_{c,i}, z_{f,j})$ spans $[r_{f,i}, r_{f,i+1}] \times [z_{c,j-1}, z_{c,j}]$.

AB2 predictor step (including the known pressure gradient):

$$w_{i,j}^* = w_{i,j}^n + \Delta t \left[\frac{3}{2} F_{w,i,j}^n - \frac{1}{2} F_{w,i,j}^{n-1} - \frac{p_{i,j}^n - p_{i,j-1}^n}{\Delta z} \right], \quad F_w = -C_r^w - C_z^w + D_r^w + D_z^w \quad (27)$$

where the convective terms (upwind) are:

$$C_r^w = \frac{r_{i+\frac{1}{2}} u_E w_{E,\text{up}} - r_{i-\frac{1}{2}} u_I w_{I,\text{up}}}{r_{c,i} \Delta r} \quad (28)$$

$$C_z^w = \frac{\bar{w}_N w_{N,\text{up}} - \bar{w}_S w_{S,\text{up}}}{\Delta z} \quad (29)$$

and the diffusive terms (central) are:

$$D_r^w = \frac{1}{\text{Re}} \frac{r_{i+\frac{1}{2}} (w_{i+1,j} - w_{i,j}) - r_{i-\frac{1}{2}} (w_{i,j} - w_{i-1,j})}{r_{c,i} \Delta r^2} \quad (30)$$

$$D_z^w = \frac{1}{\text{Re}} \frac{w_{i,j+1} - 2w_{i,j} + w_{i,j-1}}{\Delta z^2} \quad (31)$$

The radial momentum equation is discretized analogously at u-face control volumes.

3.3 Temporal Discretization

Explicit second-order Adams–Bashforth time stepping is used. For a spatial operator F , the update is

$$F^{AB2} = \frac{3}{2} F^n - \frac{1}{2} F^{n-1}, \quad (32)$$

$$\phi^{n+1} = \phi^n + \Delta t F^{AB2}. \quad (33)$$

The first step uses forward Euler because F^{n-1} is not yet available. The current momentum and temperature RHS arrays are stored for the following step. AB2 needs one RHS evaluation and one pressure-Poisson solve per time step.

Why AB2 is second order. Forward Euler, $\phi^{n+1} = \phi^n + \Delta t F^n$, freezes the right-hand side at its value at t^n and so approximates the average slope over $[t^n, t^{n+1}]$ with $\mathcal{O}(\Delta t)$ error, giving local truncation error $\mathcal{O}(\Delta t^2)$ per step and global error $\mathcal{O}(\Delta t)$. AB2 instead linearly extrapolates the two most recent slopes. Expanding $F^{n-1} = F(t^n - \Delta t) = F^n - \Delta t \dot{F}^n + \frac{1}{2} \Delta t^2 \ddot{F}^n + \mathcal{O}(\Delta t^3)$,

$$\frac{3}{2} F^n - \frac{1}{2} F^{n-1} = F^n + \frac{1}{2} \Delta t \dot{F}^n - \frac{1}{4} \Delta t^2 \ddot{F}^n + \mathcal{O}(\Delta t^3), \quad (34)$$

which matches the true average slope on $[t^n, t^{n+1}]$, $F^n + \frac{1}{2} \Delta t \dot{F}^n + \mathcal{O}(\Delta t^2)$, through the $\mathcal{O}(\Delta t)$ term: the leading error in the slope estimate is pushed from $\mathcal{O}(\Delta t^0)$ (Euler) to $\mathcal{O}(\Delta t^2)$. Multiplying by Δt in the update gives local truncation error $\mathcal{O}(\Delta t^3)$ per step; accumulating over the $N = \mathcal{O}(1/\Delta t)$ steps needed to reach a fixed final time gives global error $\mathcal{O}(\Delta t^2)$, i.e. AB2 is second order. Because the pressure-Poisson equation is solved directly (Section 3.5, no iterative truncation) at every step, it introduces no additional error beyond machine precision; combined with the $\mathcal{O}(\Delta x^2)$ spatial operators above, the overall scheme is therefore second order in *both* space and time, as the assignment requires.

The temperature equation uses the same AB2 extrapolation of its convection–diffusion operator.

The time step $\Delta t = 0.002$ satisfies the stability constraints:

- Convective CFL: $\Delta t / \Delta z = 0.01 \ll 1$
- Viscous stability: $\Delta t (1/\text{Re}) (1/\Delta r^2 + 1/\Delta z^2) \approx 0.2 < 0.5$

3.4 Pressure Correction Method (Projection)

A segregated approach is used. Each time step consists of:

1. **Predictor:** Extrapolate the convective–diffusive momentum RHS with AB2 and include the known pressure gradient:

$$\vec{u}^* = \vec{u}^n + \Delta t \left[\frac{3}{2} F^n - \frac{1}{2} F^{n-1} - \nabla p^n \right]. \quad (35)$$

2. **Poisson equation:** Find pressure correction p' such that the corrected velocity is divergence-free:

$$\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial p'}{\partial r} \right) + \frac{\partial^2 p'}{\partial z^2} = \frac{1}{\Delta t} \left[\frac{1}{r} \frac{\partial (r u^*)}{\partial r} + \frac{\partial w^*}{\partial z} \right] \quad (36)$$

Discretized:

$$\frac{r_{i+\frac{1}{2}}(p'_{i+1,j} - p'_{i,j}) - r_{i-\frac{1}{2}}(p'_{i,j} - p'_{i-1,j})}{r_{c,i} \Delta r^2} + \frac{p'_{i,j+1} - 2p'_{i,j} + p'_{i,j-1}}{\Delta z^2} = b_{i,j} \quad (37)$$

3. **Velocity correction:**

$$u_{i,j}^{n+1} = u_{i,j}^* - \Delta t \frac{p'_{i,j} - p'_{i-1,j}}{\Delta r} \quad (38)$$

$$w_{i,j}^{n+1} = w_{i,j}^* - \Delta t \frac{p'_{i,j} - p'_{i,j-1}}{\Delta z} \quad (39)$$

4. **Pressure update** with under-relaxation:

$$p^{n+1} = p^n + \alpha_p p', \quad \alpha_p = 0.7 \quad (40)$$

3.5 Poisson Solver

The Poisson equation (36) is assembled into a sparse linear system $\mathbf{A} \mathbf{p}' = \mathbf{b}$ of size $(n_r \cdot n_z) \times (n_r \cdot n_z)$. The matrix $\mathbf{A}$ is assembled once and reused every time step. It is solved using MATLAB's built-in **direct sparse solver** (backslash operator $\mathbf{A} \backslash \mathbf{b}$), which employs LU decomposition for the sparse system.

Boundary conditions for p' :

- Centreline ($r = 0$): Neumann $\partial p' / \partial r = 0$ (natural: $r_{-\frac{1}{2}} = 0$, flux vanishes)
- Wall ($r = 0.5$): Neumann $\partial p' / \partial r = 0$
- Inlet ($z = 0$): Neumann $\partial p' / \partial z = 0$
- Outlet ($z = 50$): Dirichlet $p' = 0$ is imposed at the last pressure cell centre as the reference. This location is half a cell upstream of the physical outlet face, so the condition is a discrete pressure reference rather than an exactly face-centred outlet condition.

3.6 Ghost Cells and Boundary Implementation

Ghost cells are used to impose the velocity and temperature boundary conditions in the staggered grid. Pressure-correction boundary conditions are imposed in the sparse Poisson matrix; the outlet reference is applied at the last pressure cell centre rather than through a pressure ghost cell.

Table 1: Ghost cell values for each boundary.

Boundary	u_{ghost}	w_{ghost}	p_{ghost}	θ_{ghost}
Centreline ($r = 0$)	$u_g = 0$	$w_g = w_1$ (symmetry)	$p_g = p_1$	$\theta_g = \theta_1$
Wall ($r = 0.5$)	$u_g = 0$	$w_g = -w_{n_r}$ (no-slip)	$p_g = p_{n_r}$	$\theta_g = \theta_{n_r} + \Delta r \cdot \text{RePr}$
Inlet ($z = 0$)	$u_g = -u_1$	$w = 1$ (Dirichlet)	$p_g = p_1$	$\theta = 0$ (Dirichlet)
Outlet ($z = L$)	$u_g = u_{n_z}$	$w_g = w_{n_z}$	$p' = 0$	$\theta_g = \theta_{n_z}$

The wall temperature ghost cell $\theta_g = \theta_{n_r} + \Delta r \cdot \text{RePr}$ implements the Neumann condition $\partial \theta / \partial r = \text{RePr}$ via central differencing:

$$\left. \frac{\partial \theta}{\partial r} \right|_{\text{wall}} \approx \frac{\theta_g - \theta_{n_r}}{\Delta r} = \frac{\Delta r \cdot \text{RePr}}{\Delta r} = \text{RePr} \quad (41)$$

An earlier version of this derivation incorrectly dropped the RePr factor (giving $\partial \theta / \partial r = -1$); see `Report/temperature_formula_review.md` for the full re-derivation and cross-checks.

4 Implementation

4.1 Code Structure

The solver is implemented in MATLAB. Table 2 lists all functions and their roles.

Table 2: MATLAB function structure.

Function	Description
<code>constants.m</code>	Returns struct with physical parameters (Re, Pr), grid sizes, time step, tolerances
<code>make_grid.m</code>	Generates staggered grid: $r_c, r_f, z_c, z_f, \Delta r, \Delta z$
<code>initialise_fields.m</code>	Allocates fields (u, w, p, θ) with uniform initial conditions
<code>apply_bc_w.m</code>	Enforces axial velocity BCs (inlet Dirichlet, outlet zero-gradient)
<code>apply_bc_u.m</code>	Enforces radial velocity BCs (axis and wall: $u = 0$)
<code>apply_bc_T.m</code>	Enforces temperature BCs (inlet Dirichlet, outlet zero-gradient)
<code>apply_all_bc.m</code>	Dispatches to <code>apply_bc_u/w/T.m</code> in one call (used for the initial-condition BCs before the time loop)
<code>upwind2_face.m</code>	2nd-order upwind (linear-extrapolation) face-value helper, shared by all three RHS functions below
<code>compute_rhs_w.m</code>	Explicit RHS of axial momentum (no pressure): 2nd-order upwind convection, central diffusion
<code>compute_rhs_u.m</code>	Explicit RHS of radial momentum (no pressure): 2nd-order upwind convection, central diffusion
<code>compute_rhs_T.m</code>	Explicit RHS of the energy equation: 2nd-order upwind convection, central diffusion
<code>predictor_w.m</code>	Thin 1-stage-Euler wrapper around <code>compute_rhs_w.m</code> (kept for standalone use)
<code>predictor_u.m</code>	Thin 1-stage-Euler wrapper around <code>compute_rhs_u.m</code> (kept for standalone use)
<code>build_poisson_matrix.m</code>	Assembles sparse Poisson matrix $\mathbf{A}$ (called once)
<code>solve_poisson.m</code>	Solves $\mathbf{A} \mathbf{p}' = \mathbf{b}$ using MATLAB backslash
<code>correct_velocity.m</code>	Applies pressure-gradient correction to u^*, w^*
<code>update_pressure.m</code>	Updates pressure: $p^{n+1} = p^n + \alpha_p p'$
<code>advance_temperature.m</code>	Thin 1-stage-Euler wrapper around <code>compute_rhs_T.m</code> (kept for standalone use)
<code>ab2_step.m</code>	Advances (u, w, p, θ) with AB2 and one incremental pressure correction; stores the current RHS for the next step
<code>compute_divergence.m</code>	Computes discrete divergence of velocity field
<code>compute_residuals.m</code>	Computes convergence residuals ($R_{\text{cont}}, R_{\text{vel}}, R_{\theta}$)
<code>check_convergence.m</code>	Returns true when all residuals are below tolerances
<code>analytical_velocity.m</code>	Hagen–Poiseuille velocity profile
<code>analytical_temperature.m</code>	Analytical temperature profile for constant wall heat flux
<code>main.m</code>	Main script: initialisation $\rightarrow$ time stepping $\rightarrow$ post-processing and plotting

4.2 Algorithm Flowchart

Figure 3 shows the algorithm implemented in `main.m`.

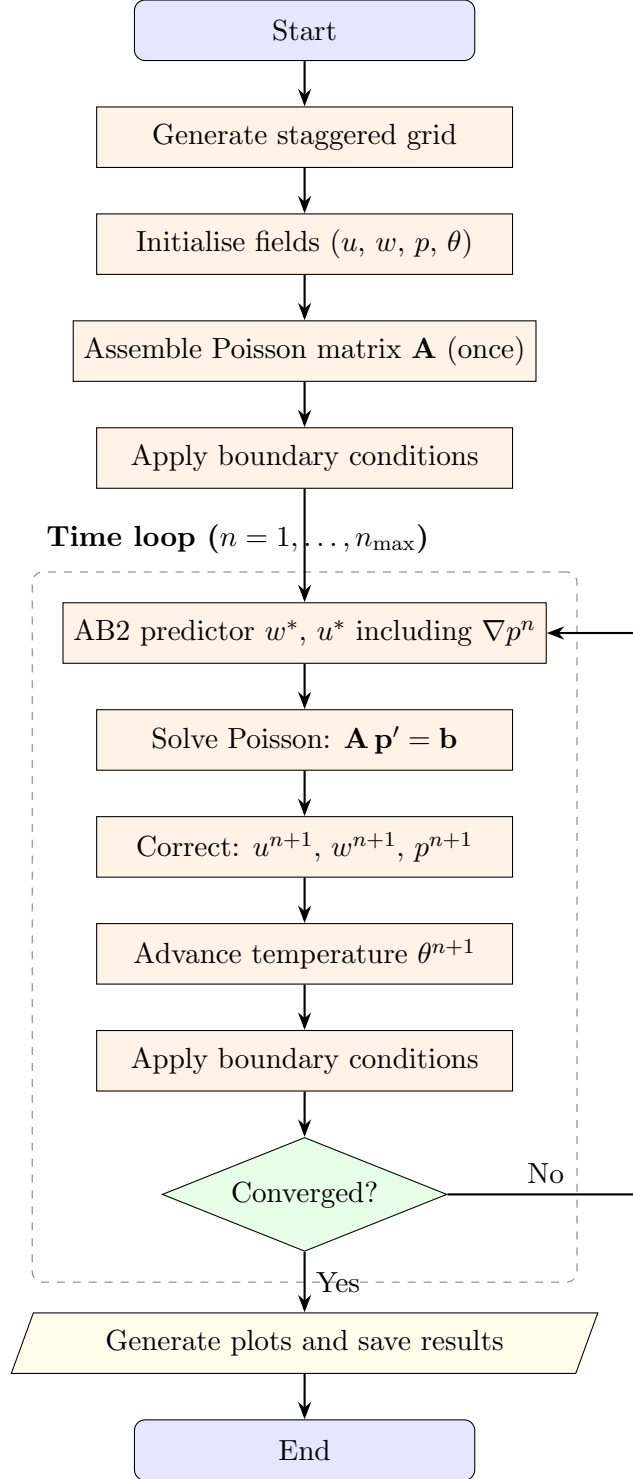


Figure 3: Algorithm flowchart for the pipe flow solver.

4.3 Key Implementation Details

- **2nd-order upwind differencing:** Convective fluxes in both momentum and energy equations use LUD. It is formally second-order on smooth, resolved regions,

as required by the assignment, but it is unbounded and can produce small local overshoots near sharp transients or boundaries.

- **Sparse direct solver:** The Poisson matrix ($12,500 \times 12,500$) is assembled once using MATLAB's `sparse` function and solved each time step with the backslash operator (`A\b`).
- **Vectorised operations:** All stencil computations are performed using MATLAB's array operations, avoiding explicit loops over grid cells for efficiency.
- **Staggered grid:** Prevents checkerboard pressure oscillations; velocity components are stored at the faces where mass fluxes are naturally evaluated.
- **Ghost cells:** Velocity and temperature boundary conditions are implemented through ghost-cell extensions. Pressure-correction boundary conditions are imposed directly in the sparse Poisson matrix, including the last-cell-centre outlet reference.

5 Analytical Solution

For fully developed flow (large z), analytical solutions exist for comparison.

5.1 Velocity Profile (Hagen–Poiseuille)

$$\frac{w}{W_{\text{in}}} = 2 [1 - (2r^*)^2] \quad (42)$$

The centreline velocity is $w_{\text{max}} = 2 W_{\text{in}}$. The hydrodynamic entry length is approximately $L_h \approx 0.05 \text{ Re } D = 5D$.

5.2 Temperature Profile

For constant wall heat flux in fully developed thermal flow:

$$\theta = 4z^* + \text{RePr} \left[\frac{1}{2}(2r^*)^2 - \frac{1}{8}(2r^*)^4 - \frac{7}{48} \right] \quad (43)$$

This form follows from direct integration of the non-dimensional energy equation (18) in the fully developed regime (Section 5.4), subject to the wall condition $\lambda \partial T / \partial r|_{\text{wall}} = q_w$ and the global energy balance $\theta_{\text{bulk}}(z^*) = 4z^*$; see Section 5.4 for the full derivation and literature sources.

The thermal entry length is $L_t \approx 0.05 \text{ Re Pr } D = 25D$.

5.3 Validity of the Fully-Developed Temperature Profile

Equation (43) makes two logically separate claims, which hold over different ranges of z^* because they rest on different physics: one about the cross-sectional *level* of the temperature (the bulk value), and one about its *radial shape*.

The bulk temperature is exact from the inlet. The bulk (mixing-cup) temperature is the flow-weighted cross-sectional average

$$\theta_{\text{bulk}}(z^*) = \frac{\int w^* \theta \, dA}{\int w^* \, dA}. \quad (44)$$

Its axial evolution follows from an energy balance on a thin slice of pipe — enthalpy gained by the stream equals heat entering through the wall:

$$\dot{m} c_v \frac{\partial T_{\text{bulk}}}{\partial z} = q_w \pi D. \quad (45)$$

Every factor on the right is constant (q_w is constant and applied from $z = 0$; the perimeter is fixed), and $\dot{m}$, c_v are constant, so $\partial T_{\text{bulk}} / \partial z$ is constant along the *entire* pipe. Non-dimensionalized this is $\partial \theta_{\text{bulk}} / \partial z^* = 4$, and integrating from $\theta_{\text{bulk}}(0) = 0$ gives

$$\theta_{\text{bulk}}(z^*) = 4z^* \quad \text{exactly, for all } z^* \geq 0, \quad (46)$$

developed or not. This balance involves only totals (total wall heat input, total enthalpy flux); it never refers to the shape of the radial profile, so the thermal entrance region does not perturb it. The one effect that would — axial conduction near the inlet — is neglected here, as specified.

The radial shape requires thermally developed flow. Reducing the energy equation (18) to the closed r^* -form in (43) assumes the profile is *thermally fully developed*: self-similar in shape, merely translating downstream, so $\partial(\theta - \theta_{\text{bulk}})/\partial z^* = 0$. That is exactly the step that makes $\partial\theta/\partial z^*$ the r^* -independent constant 4 and collapses the PDE to an ODE in r^* . Near the inlet it is false: at $z^* = 0$ the temperature is uniform ($\theta = 0$, flat); the developed shape has the wall hotter than the centreline by $\frac{11}{48}\text{RePr}$; and over the thermal entry length $L_t \approx 25D$ the profile morphs continuously between the two, a thin heated layer growing inward from the wall while the core stays at inlet temperature. Through this region $\partial(\theta - \theta_{\text{bulk}})/\partial z^*$ is large and r^* -dependent, and the ODE solution does not apply. Evaluated there anyway, (43) gives at the centreline $4z^* + \text{RePr}(-\frac{7}{48}) \approx 4z^* - 73$, i.e. negative for $z^* \lesssim 18$ — claiming the core is colder than the inlet. The numerical solution instead holds the core near $\theta = 0$ until the thermal layer reaches it ($z^* \approx 25$), then bends toward the analytical curve, the two nearly coinciding by $z^* = 50$ (Fig. 9). By contrast the hydrodynamic entry length is only $L_h \approx 0.05 \text{Re} D \approx 5D$: velocity develops about five times sooner, since here momentum diffuses $\text{Pr} = 5$ times faster than heat.

Why the level can be right while the shape is not. The constant $-\frac{7}{48}$ is fixed by requiring the flow-weighted average of the radial bracket to vanish, so (43) is *built* to satisfy $\theta_{\text{bulk}} = 4z^*$; its mean is pinned to the correct value even where its distribution about that mean is wrong. This mirrors the Hagen–Poiseuille profile (42), which always carries the correct flow rate (mean $w^* = 1$ by construction) although the true profile near the inlet is plug-like, not parabolic — right integral, wrong shape. The fully-developed profile is therefore used only as the $z^* \rightarrow \infty$ reference it is: its departure from the numerical solution at small z^* is expected behaviour, not a defect, and resolving the entrance region is the numerical solution’s role.

5.4 Nusselt Number

For hydrodynamically and thermally fully developed laminar flow in a circular pipe with *constant wall heat flux*, the Nusselt number is the exact rational value

$$\text{Nu} = \frac{hD}{\lambda} = \frac{q_w D}{\lambda(T_{\text{wall}} - T_{\text{bulk}})} = \frac{\text{RePr}}{\theta_{\text{wall}} - \theta_{\text{bulk}}} = \frac{48}{11} \approx 4.3636, \quad (47)$$

a standard textbook result for this thermal boundary condition [1, 2].

Derivation. In the fully developed region $u^* = 0$ and the profile translates rigidly in z^* , so the non-dimensional energy equation (18) reduces to

$$w^*(r^*) \frac{\partial\theta}{\partial z^*} = \frac{1}{\text{RePr}} \frac{1}{r^*} \frac{\partial}{\partial r^*} \left(r^* \frac{\partial\theta}{\partial r^*} \right). \quad (48)$$

A global energy balance over a pipe slice fixes the axial gradient: with the wall adding heat at non-dimensional rate $\partial\theta/\partial r^*|_{\text{wall}} = \text{RePr}$ and the flow carrying mean velocity $\langle w^* \rangle = 1$, the bulk (mixing-cup) temperature satisfies $\theta_{\text{bulk}}(z^*) = 4z^*$ exactly, hence $\partial\theta/\partial z^* = 4$ everywhere in the cross-section. Substituting this and $w^* = 2[1 - (2r^*)^2]$ into (48) and integrating twice, with $\partial\theta/\partial r^*|_{r^*=0} = 0$ and $\partial\theta/\partial r^*|_{r^*=1/2} = \text{RePr}$,

$$\theta(r^*, z^*) = 4z^* + \text{RePr} \left[\frac{1}{2}(2r^*)^2 - \frac{1}{8}(2r^*)^4 + c \right], \quad (49)$$

and the constant c follows from requiring the flow-weighted average of the bracket to vanish (so that $\theta_{\text{bulk}} = 4z^*$), which gives $c = -\frac{7}{48}$ — recovering Eq. (43). Evaluating the bracket at the wall gives $\theta_{\text{wall}} - \theta_{\text{bulk}} = \frac{11}{48} \text{RePr}$, and (47) follows.

Consistency check. The value is exact: (48) with its boundary conditions is a linear problem with no free parameters, so Nu is independent of Re , Pr , q_w , the fluid properties and the grid. Taking instead the assignment sheet’s literal radial coefficients $(2r^*)^2 - \frac{1}{4}(2r^*)^4 - \frac{3}{4}$ yields $\theta_{\text{wall}} - \theta_{\text{bulk}} = \frac{11}{24} \text{RePr}$ and hence $\text{Nu} = 24/11 \approx 2.18$ — exactly half of $48/11$, and inconsistent both with this classical result and with the assignment’s own stated wall condition (that literal profile has wall gradient 2RePr , and its bulk temperature is not $4z^*$).

Empirical check against the CFD solution. The solver imposes the physically consistent wall condition $\partial\theta/\partial r^* = \text{RePr}$. Computing the flow-weighted bulk temperature from the converged field ($t^* = 139.728$) and forming $\text{Nu} = \text{RePr}/(\theta_{\text{wall}} - \theta_{\text{bulk}})$ at three downstream stations:

z^*	settling margin $t^* - z^*$	θ_{bulk} (CFD)	$4z^*$ (expected)	Nu (CFD)	classical $48/11$
29.9	30.1	119.43	119.6	4.585	4.364
39.9	20.1	155.82	159.6	4.727	4.364
48.9	11.1	181.79	195.6	5.048	4.364

The global balance $\theta_{\text{bulk}} \approx 4z^*$ holds to $\sim 1\%$ at $z^* = 29.9$ (largest settling margin), and the CFD Nu is within $\sim 5\%$ of $48/11$ there, degrading toward the outlet exactly as expected from progressively less settling time (thermal diffusion across the radius has non-dimensional timescale $\sim \text{RePr}R^{*2} \approx 125$, comparable to the simulated horizon). This is a model-free confirmation, built only from the governing PDE and the wall BC, that the physical answer is near $48/11$ and not near the 2.18 implied by the assignment-literal coefficients.

Why the solver can never reproduce $24/11$. The energy equation, the wall BC, and the inlet condition $\theta_{\text{in}} = 0$ are all *linear* in θ , and Nu in (47) is normalised by the wall flux. Multiplying the imposed wall flux by any constant k simply scales the entire θ field by k , leaving Nu unchanged at $48/11$. A CFD run therefore cannot reproduce $24/11$: that number exists only as a property of the internally inconsistent literal formula, never of a correctly posed simulation [1, 2, 3, 4].

6 Results

6.1 Simulation Parameters

Table 3: Simulation parameters.

Parameter	Value	Description
Re	100	Reynolds number
Pr	5	Prandtl number
$n_r \times n_z$	50×250	Grid resolution
Δr	0.01	Radial grid spacing
Δz	0.2	Axial grid spacing
Δt	0.002	Time step
n_{steps}	30 000	Maximum time steps
α_p	0.7	Pressure under-relaxation
Convective scheme	2nd-order upwind (LUD)	Stability for $Pe \gg 2$; formally 2nd-order on smooth
Time integration	Explicit Adams–Bashforth 2	2nd-order; Euler startup

6.2 Convergence History

The simulation targets convergence when the continuity, velocity, and temperature residuals fall below 10^{-6} :

$$R_{\text{cont}} = \max_{i,j} |\nabla \cdot \mathbf{u}|, \quad R_{\text{vel}} = \frac{1}{\Delta t} \max(\max |u^{n+1} - u^n|, \max |w^{n+1} - w^n|), \quad R_\theta = \frac{\max |\theta^{n+1} - \theta^n|}{\Delta t} \quad (50)$$

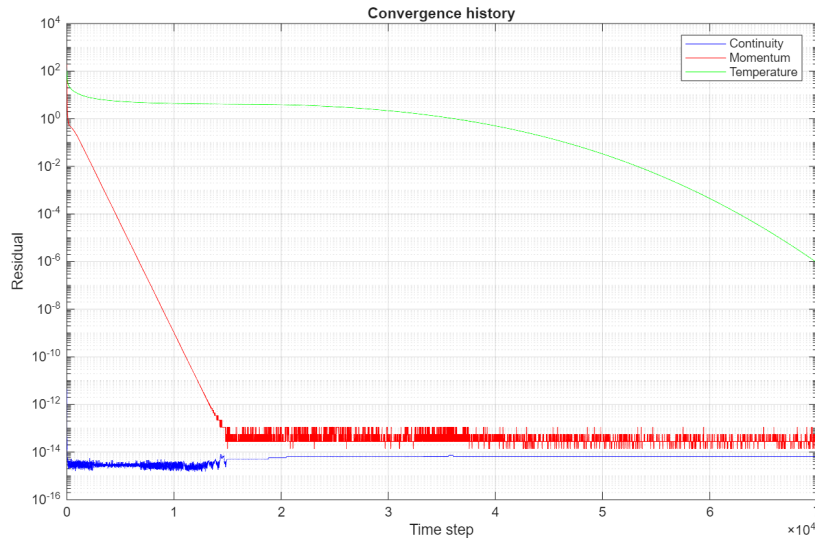


Figure 4: Convergence history: residuals versus time step.

The continuity residual (blue) is flat from the very first step rather than decaying over time: the incremental pressure correction re-solves the Poisson equation to machine precision every single step, so $\nabla \cdot \mathbf{u}$ is driven to its numerical floor ($\sim 10^{-14}$ – 10^{-15}) at each

iteration by construction, independent of whether the flow itself has reached steady state. R_{vel} (red), in contrast, measures how much the velocity field is still changing between steps: it decays exponentially, by roughly one decade every 1000 steps ($\Delta t^* = 2$), until it reaches the round-off floor at step $\approx 15,000$ ($t^* \approx 30$) – only then does it flatten into a noisy plateau rather than continuing smoothly to zero. That plateau is round-off noise: R_{vel} divides an already near-machine-precision difference $u^{n+1} - u^n$ by $\Delta t = 0.002$, amplifying floating-point noise by a factor $1/\Delta t = 500$, which is why it appears jagged at $\mathcal{O}(10^{-13})$ while R_{cont} (not divided by Δt) stays visually flat near $\mathcal{O}(10^{-14})$ – $\mathcal{O}(10^{-15})$.

In the extended MATLAB run, $R_{\text{cont}} = 6.46 \times 10^{-15}$ and $R_{\text{vel}} = 2.78 \times 10^{-14}$ at the converged time $t^* = 139.728$. The temperature residual decreases from approximately 101 at the first step and reaches $R_\theta = 9.99 \times 10^{-7}$ at step 69,864, just below the target 10^{-6} . This residual is the maximum local rate of change, not the error relative to the analytical temperature profile. Its earlier non-monotone decrease was therefore not a numerical failure: as the heated layer moved downstream and diffused inward, the cell containing the largest change moved as well. The dominant timescale is thermal diffusion across the radius, $\text{RePr}R^{*2} \approx 125$, which explains why the temperature required substantially longer to settle than the velocity. The final result is therefore a converged finite-volume solution at the stated residual tolerance, while the analytical fully-developed temperature profile remains a separate asymptotic reference for radial shape.

6.3 Velocity Field

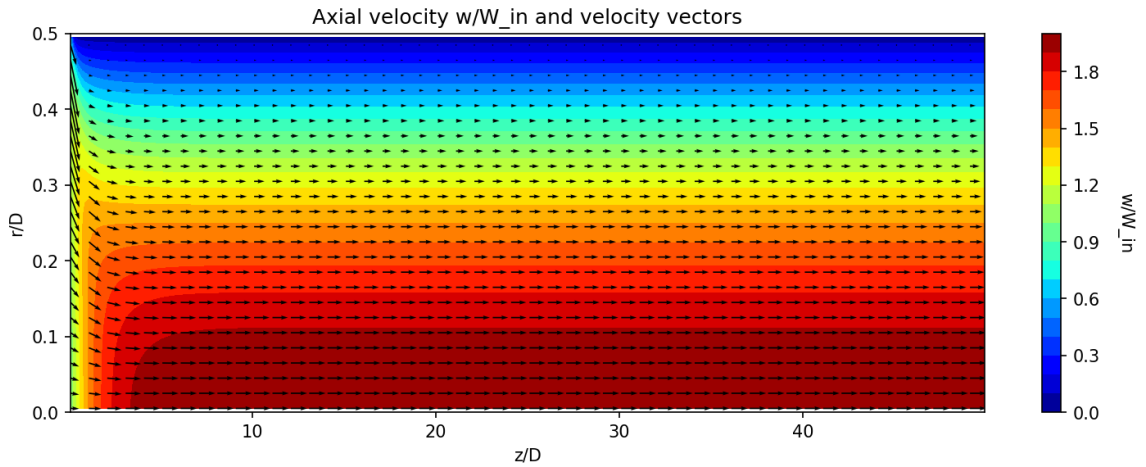


Figure 5: Axial velocity contour w/W_{in} with velocity vectors throughout the domain. The flow transitions from a uniform inlet profile ($w = 1$) to a parabolic profile ($w_{\text{max}} = 2$ at centreline); the arrows show the axial and radial components of the velocity field at regularly spaced cell centres.

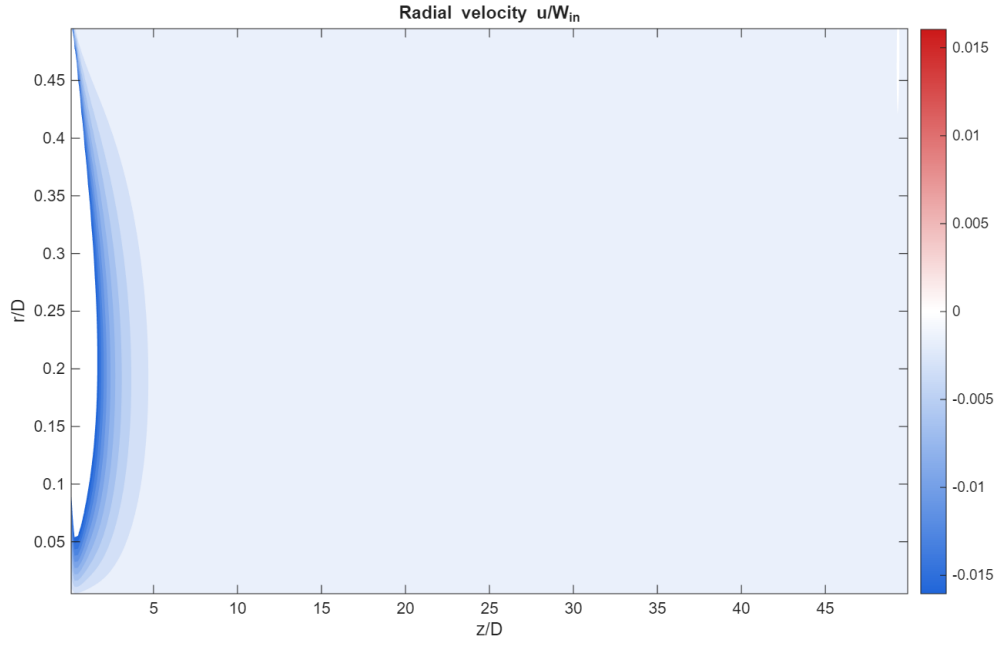


Figure 6: Radial velocity contour u/W_{in} . Non-zero only in the entrance region where the flow is developing; vanishes as the flow becomes fully developed.

6.4 Temperature Field

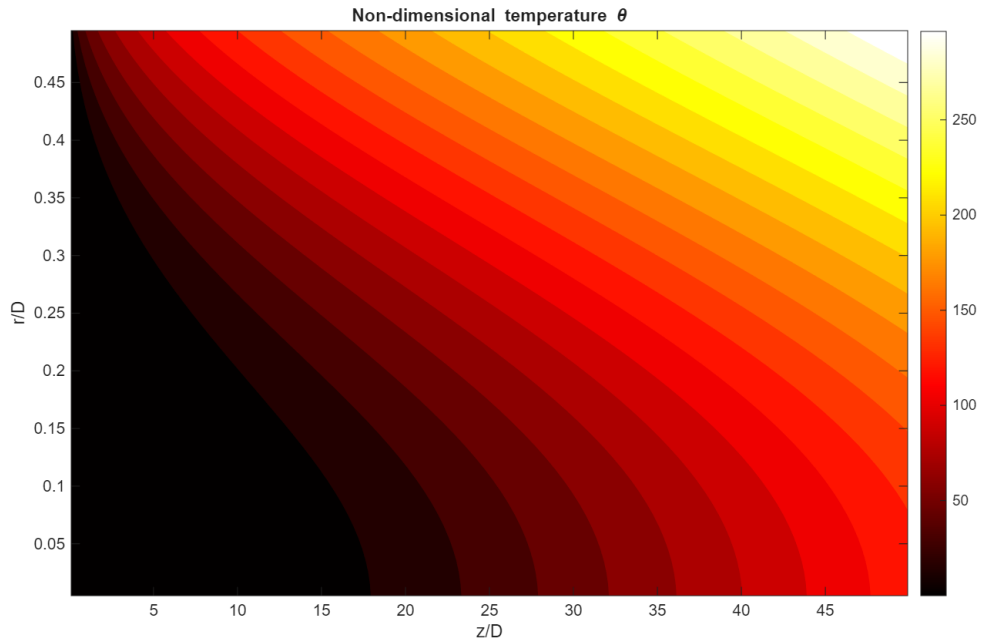


Figure 7: Non-dimensional temperature contour θ . Temperature increases along the pipe due to the constant wall heat flux.

6.5 Velocity Profiles at Selected Axial Positions

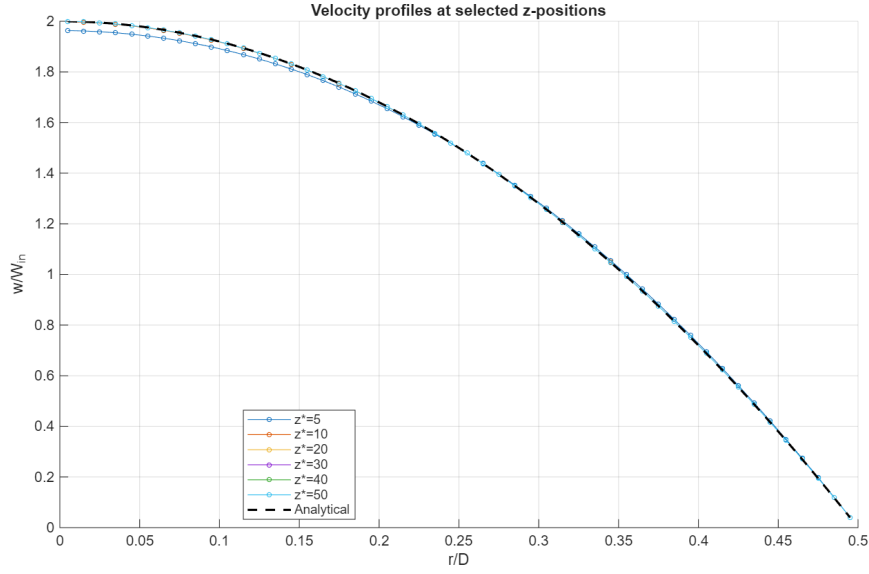


Figure 8: Axial velocity profiles $w(r)$ at $z^* = 5, 10, 20, 30, 40, 50$. Dashed lines show the analytical Hagen–Poiseuille profile. The velocity profile develops from uniform at the inlet to parabolic downstream.

6.6 Temperature Profiles at Selected Axial Positions

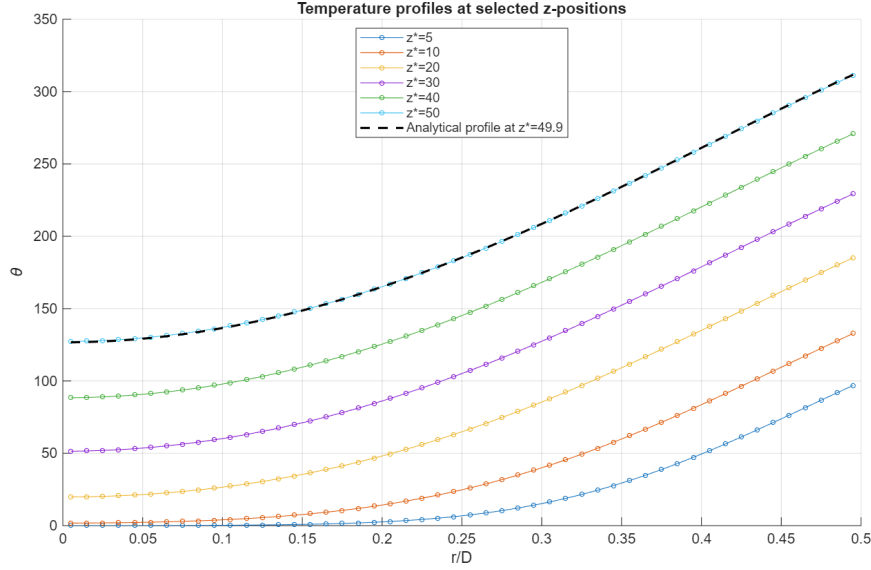


Figure 9: Non-dimensional temperature profiles $\theta(r)$ at selected axial positions (solid, numerical). The dashed curve is the analytical fully-developed profile (43), overlaid only at the largest scalar location ($z^* = 49.9$). The numerical curve has the expected developed radial shape; any remaining difference from the dashed curve reflects comparison with a fully-developed asymptotic radial profile and the finite axial location. The dashed curve is a *fully-developed asymptote* (Section 5.3): only meant to be reached for increasing z^* , not a valid prediction at every z^* — evaluated at small z^* (e.g. $z^* = 4.9$) it would extrapolate to strongly negative θ near the centreline (the $4z^*$ term not yet compensating for the formula’s negative radial offset). The final numerical field is essentially non-negative; a small, localized transient LUD undershoot occurs near the inlet–wall corner during the first time steps. Hence the asymptotic curve is not plotted at those smaller- z^* positions.

6.7 Comparison with Analytical Solution

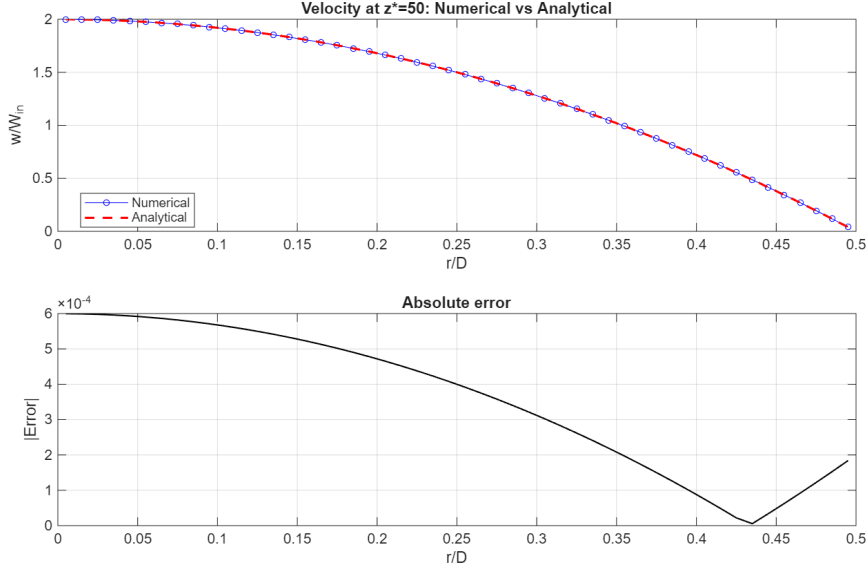


Figure 10: Velocity profile at the last cell-centre location, $z^* = 49.9$ (nominally the $z^* = 50$ outlet, fully developed): numerical vs. analytical Hagen–Poiseuille solution. The absolute-error minimum near $r^* \approx 0.43$ occurs where the signed difference between the numerical and analytical profiles changes sign. Conservation fixes the numerical area-mean velocity through the imposed inlet flow rate, which agrees with the continuous Hagen–Poiseuille mean; consequently, a small negative core error is compensated by a small positive near-wall error. The discrete cell-centre means differ slightly because of radial quadrature. The subsequent rise toward the wall is associated with local wall and cylindrical-curvature discretization, wall ghost-cell treatment, and profile sampling, not with a physical velocity minimum.

7 Conclusion

A finite volume solver for 2D axisymmetric incompressible pipe flow was developed in MATLAB and validated against analytical solutions. The solver uses a staggered grid with 2nd-order upwind (linear upwind) differencing for convective stability, explicit second-order Adams–Bashforth time integration, and an incremental projection method with one sparse direct Poisson solve per time step.

The numerical results reproduce the expected physics:

- The velocity field develops from a uniform inlet profile to the parabolic Hagen–Poiseuille profile within the hydrodynamic entry length. At $z^* = 50$ the numerical and analytical Hagen–Poiseuille profiles agree to within $\mathcal{O}(10^{-4})$ (Fig. 10) – the velocity field is essentially fully converged and validated.
- The temperature field shows the characteristic linear rise of the bulk temperature along the pipe axis ($\theta_{\text{bulk}} \approx 4z^*$, confirmed to within $\sim 1\%$ at $z^* = 29.9$; see Section 5.4) and a radial profile that approaches the analytical fully-developed solution as z^* increases (Fig. 9), as expected since that solution is only valid asymptotically. Agreement is closest where the flow has had the most settling time; at smaller z^* the difference is primarily the thermal entrance-region shape, while the final field satisfies the temporal convergence criterion.

A 2D Incompressible Flow in Cylindrical Coordinates

(r, z) — Velocities (u, w)

Note on notation: this appendix derives the FVM discretisation, projection method, and ghost-cell treatment in general form, illustrating u and w as cell-centred quantities near each boundary for schematic clarity. The staggered grid actually implemented (Table 2 in the main text) instead stores u exactly *at* the axis and wall faces and w exactly *at* the inlet and outlet faces, so those boundary-normal components are set directly (Dirichlet) rather than via a mirrored ghost cell; only the wall-tangential component and the scalars (p, θ) use the mirrored ghost-cell form shown here. The relaxation factor ζ below is the same quantity as α_p in Section 3.4, where this submission uses $\alpha_p = 0.7$.

A.1 Governing Equations (Strong Form)

For axisymmetric flow ($\partial/\partial\phi = 0$, $v_\phi = 0$) in cylindrical coordinates (r, z) :

Continuity:

$$\frac{1}{r} \frac{\partial(ru)}{\partial r} + \frac{\partial w}{\partial z} = 0 \quad (51)$$

Radial momentum (r -direction):

$$\frac{\partial u}{\partial t} + \frac{1}{r} \frac{\partial(ru^2)}{\partial r} + \frac{\partial(uw)}{\partial z} = -\frac{1}{\rho} \frac{\partial p}{\partial r} + \frac{1}{\rho r} \frac{\partial(r\tau_{rr})}{\partial r} + \frac{1}{\rho} \frac{\partial\tau_{rz}}{\partial z} \quad (52)$$

Axial momentum (z -direction):

$$\frac{\partial w}{\partial t} + \frac{1}{r} \frac{\partial(ruw)}{\partial r} + \frac{\partial(w^2)}{\partial z} = -\frac{1}{\rho} \frac{\partial p}{\partial z} + \frac{1}{\rho r} \frac{\partial(r\tau_{rz})}{\partial r} + \frac{1}{\rho} \frac{\partial\tau_{zz}}{\partial z} \quad (53)$$

where for a Newtonian fluid with constant viscosity $\mu = \rho\nu$:

$$\tau_{rr} = 2\mu \frac{\partial u}{\partial r} \quad (54)$$

$$\tau_{zz} = 2\mu \frac{\partial w}{\partial z} \quad (55)$$

$$\tau_{rz} = \mu \left(\frac{\partial u}{\partial z} + \frac{\partial w}{\partial r} \right) \quad (56)$$

A.2 Staggered Grid Layout

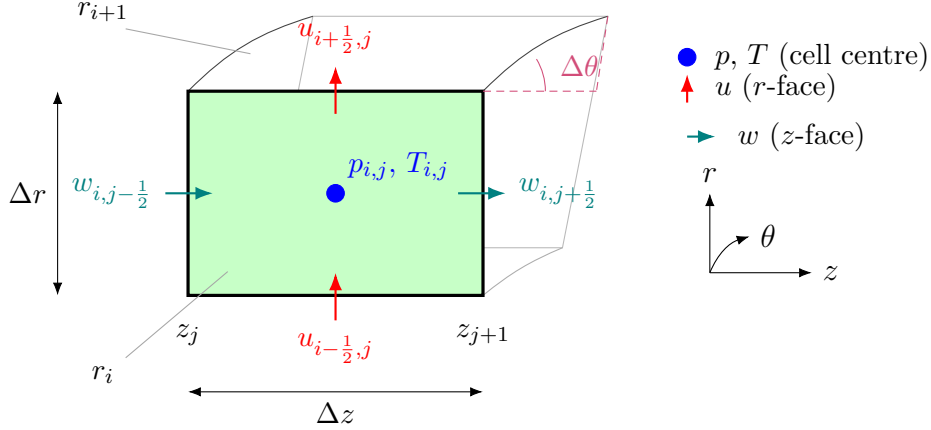


Figure 11: Staggered grid in the (r, z) plane. Scalars (p, T) at the cell centre (blue); radial velocity u on the (curved, cylindrical) r -faces (red); axial velocity w on the z -faces (teal). The control volume is drawn swept through a finite $\Delta\theta$ purely to convey the 3-D cylindrical shape of the r -faces (note the outer face at r_{i+1} sweeps further than the inner face at r_i , since arc length = $r \Delta\theta$); the axisymmetric problem itself has $\partial/\partial\theta = 0$ and no θ -dependence.

A.3 Finite Volume Discretisation — Integration over $\Delta r \Delta z$

The volume element for an axisymmetric ring CV is $dV = r dr d\phi dz$. After integrating over $\phi \in [0, 2\pi]$, every integral acquires the factor 2π . Since the same factor appears on both sides of the momentum equations, it cancels and we work with the *area* element $r dr dz$.

A.3.1 Continuity Equation

Integrate Eq. (51) over the scalar CV $[r_{i-\frac{1}{2}}, r_{i+\frac{1}{2}}] \times [z_{j-\frac{1}{2}}, z_{j+\frac{1}{2}}]$:

$$\int_{r_{i-\frac{1}{2}}}^{r_{i+\frac{1}{2}}} \int_{z_{j-\frac{1}{2}}}^{z_{j+\frac{1}{2}}} \left[\frac{1}{r} \frac{\partial(ru)}{\partial r} + \frac{\partial w}{\partial z} \right] r dr dz = 0 \quad (57)$$

Applying the FV divergence theorem:

$$\boxed{\frac{r_{i+\frac{1}{2}} u_{i+\frac{1}{2},j} - r_{i-\frac{1}{2}} u_{i-\frac{1}{2},j}}{r_i \Delta r} + \frac{w_{i,j+\frac{1}{2}} - w_{i,j-\frac{1}{2}}}{\Delta z} = 0} \quad (58)$$

A.3.2 Axial Momentum: CV at $(i, j + \frac{1}{2})$

Integrate Eq. (53) over $[r_{i-\frac{1}{2}}, r_{i+\frac{1}{2}}] \times [z_j, z_{j+1}]$, weighting by r :

Temporal term:

$$\frac{\partial w_{i,j+\frac{1}{2}}}{\partial t} \approx \frac{w_{i,j+\frac{1}{2}}^{n+1} - w_{i,j+\frac{1}{2}}^n}{\Delta t} \quad (59)$$

Convective terms:

$$u \frac{\partial w}{\partial r} \Big|_{i,j+\frac{1}{2}} \approx \bar{u}_{i,j+\frac{1}{2}} \frac{w_{i+1,j+\frac{1}{2}} - w_{i-1,j+\frac{1}{2}}}{2\Delta r} \quad (60)$$

$$w \frac{\partial w}{\partial z} \Big|_{i,j+\frac{1}{2}} \approx w_{i,j+\frac{1}{2}} \frac{w_{i,j+\frac{3}{2}} - w_{i,j-\frac{1}{2}}}{2\Delta z} \quad (61)$$

where $\bar{u}_{i,j+\frac{1}{2}} = \frac{1}{4}(u_{i+\frac{1}{2},j} + u_{i-\frac{1}{2},j} + u_{i+\frac{1}{2},j+1} + u_{i-\frac{1}{2},j+1})$.

Pressure gradient:

$$\frac{1}{\rho} \frac{\partial p}{\partial z} \Big|_{i,j+\frac{1}{2}} \approx \frac{1}{\rho} \frac{p_{i,j+1} - p_{i,j}}{\Delta z} \quad (62)$$

Viscous diffusion (using $\frac{1}{r} \frac{\partial}{\partial r}(r \frac{\partial w}{\partial r})$):

$$\nu \left(\frac{\partial^2 w}{\partial r^2} + \frac{1}{r} \frac{\partial w}{\partial r} \right) \approx \frac{\nu}{r_i \Delta r} \left(r_{i+\frac{1}{2}} \frac{w_{i+1,j+\frac{1}{2}} - w_{i,j+\frac{1}{2}}}{\Delta r} - r_{i-\frac{1}{2}} \frac{w_{i,j+\frac{1}{2}} - w_{i-1,j+\frac{1}{2}}}{\Delta r} \right) \quad (63)$$

$$\nu \frac{\partial^2 w}{\partial z^2} \approx \nu \frac{w_{i,j+\frac{3}{2}} - 2w_{i,j+\frac{1}{2}} + w_{i,j-\frac{1}{2}}}{\Delta z^2} \quad (64)$$

A.3.3 Radial Momentum: CV at $(i + \frac{1}{2}, j)$

Integrate Eq. (52) over $[r_i, r_{i+1}] \times [z_{j-\frac{1}{2}}, z_{j+\frac{1}{2}}]$:

Convective terms:

$$u \frac{\partial u}{\partial r} \Big|_{i+\frac{1}{2},j} \approx u_{i+\frac{1}{2},j} \frac{u_{i+\frac{3}{2},j} - u_{i-\frac{1}{2},j}}{2\Delta r} \quad (65)$$

$$w \frac{\partial u}{\partial z} \Big|_{i+\frac{1}{2},j} \approx \bar{w}_{i+\frac{1}{2},j} \frac{u_{i+\frac{1}{2},j+1} - u_{i+\frac{1}{2},j-1}}{2\Delta z} \quad (66)$$

Pressure gradient:

$$\frac{1}{\rho} \frac{\partial p}{\partial r} \Big|_{i+\frac{1}{2},j} \approx \frac{1}{\rho} \frac{p_{i+1,j} - p_{i,j}}{\Delta r} \quad (67)$$

Viscous diffusion:

$$\nu \left(\frac{\partial^2 u}{\partial r^2} + \frac{1}{r} \frac{\partial u}{\partial r} - \frac{u}{r^2} \right) \approx \frac{\nu}{r_{i+\frac{1}{2}} \Delta r} \left(r_{i+1} \frac{u_{i+\frac{3}{2},j} - u_{i+\frac{1}{2},j}}{\Delta r} - r_i \frac{u_{i+\frac{1}{2},j} - u_{i-\frac{1}{2},j}}{\Delta r} \right) - \frac{\nu u_{i+\frac{1}{2},j}}{r_{i+\frac{1}{2}}^2} \quad (68)$$

$$\nu \frac{\partial^2 u}{\partial z^2} \approx \nu \frac{u_{i+\frac{1}{2},j+1} - 2u_{i+\frac{1}{2},j} + u_{i+\frac{1}{2},j-1}}{\Delta z^2} \quad (69)$$

A.4 Projection Method

A.4.1 Step 1 — Predictor

Advance without the pressure gradient (explicit Euler or AB2):

$$\boxed{w_{i,j+\frac{1}{2}}^* = w_{i,j+\frac{1}{2}}^n + \Delta t \left[-\bar{u}^n \frac{w_{i+1,j+\frac{1}{2}}^n - w_{i-1,j+\frac{1}{2}}^n}{2\Delta r} - w_{i,j+\frac{1}{2}}^n \frac{w_{i,j+\frac{3}{2}}^n - w_{i,j-\frac{1}{2}}^n}{2\Delta z} + \nu D_r^w + \nu D_z^w \right]} \quad (70)$$

$$u_{i+\frac{1}{2},j}^* = u_{i+\frac{1}{2},j}^n + \Delta t \left[-u_{i+\frac{1}{2},j}^n \frac{u_{i+\frac{3}{2},j}^n - u_{i-\frac{1}{2},j}^n}{2\Delta r} - \bar{w}^n \frac{u_{i+\frac{1}{2},j+1}^n - u_{i+\frac{1}{2},j-1}^n}{2\Delta z} + \nu D_r^u + \nu D_z^u \right] \quad (71)$$

where the diffusion operators are:

$$D_r^w = \frac{1}{r_i \Delta r} \left(r_{i+\frac{1}{2}} \frac{w_{i+1,j+\frac{1}{2}}^n - w_{i,j+\frac{1}{2}}^n}{\Delta r} - r_{i-\frac{1}{2}} \frac{w_{i,j+\frac{1}{2}}^n - w_{i-1,j+\frac{1}{2}}^n}{\Delta r} \right), \quad D_z^w = \frac{w_{i,j+\frac{3}{2}}^n - 2w_{i,j+\frac{1}{2}}^n + w_{i,j-\frac{1}{2}}^n}{\Delta z^2} \quad (72)$$

$$D_r^u = \frac{r_{i+1}(u_{i+\frac{3}{2},j}^n - u_{i+\frac{1}{2},j}^n) - r_i(u_{i+\frac{1}{2},j}^n - u_{i-\frac{1}{2},j}^n)}{r_{i+\frac{1}{2}} \Delta r^2} - \frac{u_{i+\frac{1}{2},j}^n}{r_{i+\frac{1}{2}}^2} \quad (73)$$

$$D_z^u = \frac{u_{i+\frac{1}{2},j+1}^n - 2u_{i+\frac{1}{2},j}^n + u_{i+\frac{1}{2},j-1}^n}{\Delta z^2} \quad (74)$$

A.4.2 Step 2 — Poisson Equation for Pressure Correction p'

Enforcing the discrete continuity Eq. (58) after correction:

$$\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial p'}{\partial r} \right) + \frac{\partial^2 p'}{\partial z^2} = \frac{\rho}{\Delta t} \left(\frac{1}{r} \frac{\partial (ru^*)}{\partial r} + \frac{\partial w^*}{\partial z} \right) \quad (75)$$

Discretised form at cell (i, j) :

$$\frac{r_{i+\frac{1}{2}}(p'_{i+1,j} - p'_{i,j}) - r_{i-\frac{1}{2}}(p'_{i,j} - p'_{i-1,j})}{r_i \Delta r^2} + \frac{p'_{i,j+1} - 2p'_{i,j} + p'_{i,j-1}}{\Delta z^2} = \frac{\rho}{\Delta t} \left(\frac{r_{i+\frac{1}{2}} u_{i+\frac{1}{2},j}^* - r_{i-\frac{1}{2}} u_{i-\frac{1}{2},j}^*}{r_i \Delta r} + \frac{w_{i,j+\frac{1}{2}}^* - w_{i,j-\frac{1}{2}}^*}{\Delta z} \right) \quad (76)$$

A.4.3 Step 3 — Velocity Correction

$$w_{i,j+\frac{1}{2}}^{n+1} = w_{i,j+\frac{1}{2}}^* - \frac{\Delta t}{\rho} \frac{p'_{i,j+1} - p'_{i,j}}{\Delta z} \quad (77)$$

$$u_{i+\frac{1}{2},j}^{n+1} = u_{i+\frac{1}{2},j}^* - \frac{\Delta t}{\rho} \frac{p'_{i+1,j} - p'_{i,j}}{\Delta r} \quad (78)$$

A.4.4 Step 4 — Pressure Update

$$p_{i,j}^{n+1} = p_{i,j}^n + \zeta p'_{i,j}, \quad \zeta = \alpha_p = 0.7 \text{ (this submission)} \quad (79)$$

A.5 Poisson Solver Procedure

Rearranging Eq. (76) to isolate $p'_{i,j}$:

$$p'_{i,j} = \frac{b_{i,j} - \frac{r_{i+\frac{1}{2}} p'_{i+1,j} + r_{i-\frac{1}{2}} p'_{i-1,j}}{r_i \Delta r^2} - \frac{p'_{i,j+1} + p'_{i,j-1}}{\Delta z^2}}{-\frac{r_{i+\frac{1}{2}} + r_{i-\frac{1}{2}}}{r_i \Delta r^2} - \frac{2}{\Delta z^2}} \quad (80)$$

Gauss–Seidel / SOR iteration:

1. Set $p' = 0$ everywhere.
2. Enforce BCs: $\partial p'/\partial r = 0$ at wall & centreline; $\partial p'/\partial z = 0$ at inlet/outlet (or $p' = 0$ at outlet).
3. Sweep interior cells with Eq. (80); apply SOR: $p'^{\text{new}} = p'^{\text{old}} + \omega(p'^{\text{GS}} - p'^{\text{old}})$.
4. Converge when $\max |p'^{\text{new}} - p'^{\text{old}}| < \varepsilon$.

Direct solve: assemble sparse $\mathbf{A}$ from Eq. (76) and call `spsolve`. Note: fix one pressure value (e.g. $p'_{1,N_z} = 0$ at the outlet corner) to remove the null space.

A.6 Ghost Cells and Boundary Conditions

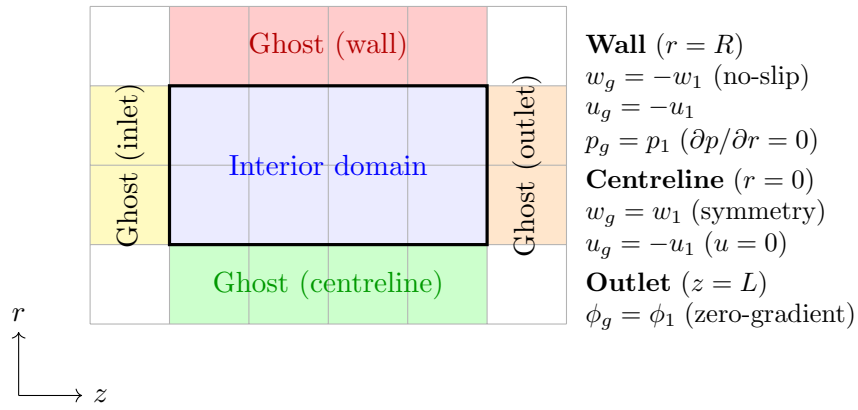


Figure 12: Ghost cell layout for the (r, z) domain. Green: centreline symmetry; red: no-slip wall; yellow: inlet; orange: outlet. Subscript g = ghost, subscript 1 = first interior cell.

Boundary	Condition	Variable	Ghost value
Centreline ($r = 0$)	$u = 0$	u	$u_g = -u_1$
	$\partial w/\partial r = 0$	w	$w_g = w_1$
	$\partial p/\partial r = 0$	p	$p_g = p_1$
Wall ($r = R$)	$u = 0, w = 0$	u	$u_g = -u_1$
		w	$w_g = -w_1$
	$\partial p/\partial r = 0$	p	$p_g = p_1$
	$\lambda \partial T/\partial r = q_w$	T	$T_g = T_1 + \frac{q_w \Delta r}{\lambda}$
Inlet ($z = 0$)	$u = 0, w = W_{\text{in}}$	w	$w_g = 2W_{\text{in}} - w_1$
		u	$u_g = -u_1$
	$T = T_{\text{in}}$	T	$T_g = 2T_{\text{in}} - T_1$
Outlet ($z = L$)	$\partial(\cdot)/\partial z = 0$	any	$\phi_g = \phi_1$

B MATLAB Source Code

The complete MATLAB source code for the AB2 solver is listed below. Each function corresponds to a module in Table 2.

B.1 Configuration and Initialisation

```
1 function c = constants()
2 % CONSTANTS Physical constants and simulation parameters.
3 % c = constants() returns a struct with all parameters.
4
5 % Non-dimensional numbers
6 c.Re = 100.0; % Reynolds number
7 c.Pr = 5.0; % Prandtl number
8
9 % Geometry (non-dimensional)
10 c.L_over_D = 50.0; % Pipe length in diameters
11
12 % Grid parameters
13 c.n_r = 50; % Radial grid cells
14 c.n_z = 250; % Axial grid cells
15
16 % Time parameters
17 c.dt = 0.002; % Time step
18 c.n_steps = 100000; % Max time steps
19 c.output_interval = 1000;
20
21 % Numerical parameters
22 c.tol_continuity = 1e-6;
23 c.tol_velocity = 1e-6;
24 c.tol_temperature = 1e-6;
25 c.alpha_p = 0.7; % Pressure under-relaxation
26 end
```

Listing 1: constants.m — Physical parameters, grid size, and solver tolerances

```
1 function [u, w, p, T] = initialise_fields(n_r, n_z)
2 % INITIALISE_FIELDS Allocate and set initial conditions.
3 %
4 % [u, w, p, T] = initialise_fields(n_r, n_z)
5 %
6 % u : (n_r+1, n_z) radial velocity at r-faces -> zero
7 % w : (n_r, n_z+1) axial velocity at z-faces -> uniform 1
8 % p : (n_r, n_z) pressure at cell centres -> zero
9 % T : (n_r, n_z) temperature at cell centres -> zero
10
11 u = zeros(n_r + 1, n_z);
12 w = ones(n_r, n_z + 1);
13 p = zeros(n_r, n_z);
14 T = zeros(n_r, n_z);
15 end
```

Listing 2: initialise_fields.m — Allocation and initial conditions for all fields

B.2 Main Simulation Script

```

1 %% MAIN Simulation loop for 2D axisymmetric incompressible pipe flow.
2 %
3 % Second-order Adams-Bashforth integration with one incremental
4 % correction per step. Forward Euler initialises the multistep history.
5
6 clear; close all; clc;
7
8 %% Load parameters
9 c = constants();
10 Re = c.Re; Pr = c.Pr;
11 n_r = c.n_r; n_z = c.n_z;
12 dt = c.dt; n_steps = c.n_steps;
13 alpha_p = c.alpha_p;
14
15 fprintf('=====\\n'
16 );
17 fprintf('Incompressible Pipe Flow Solver (MATLAB)\\n');
18 fprintf(' Re=%g, Pr=%g, n_r=%d, n_z=%d\\n', Re, Pr, n_r, n_z);
19 fprintf(' dt=%g, max steps=%d\\n', dt, n_steps);
20 fprintf('=====\\n'
21 );
22 %% Grid
23 [r_c, r_f, z_c, z_f, dr, dz] = make_grid(n_r, n_z, c.L_over_D);
24
25 % CFL check
26 CFL_conv = max(dt/dr, dt/dz);
27 CFL_visc = dt * (1/Re) * (1/dr^2 + 1/dz^2);
28 fprintf(' CFL_conv = %.4f (should be < 1)\\n', CFL_conv);
29 fprintf(' CFL_visc = %.4f (should be < 0.5)\\n', CFL_visc);
30 if CFL_visc > 0.5
31     warning('Viscous stability limit exceeded. Reduce dt or coarsen grid
32 .');
33 end
34
35 %% Initialise fields
36 [u, w, p, T] = initialise_fields(n_r, n_z);
37 [u, w, T] = apply_all_bc(u, w, T);
38
39 %% Build Poisson matrix (once)
40 fprintf('\\nAssembling Poisson matrix ...\\n');
41 A_poisson = build_poisson_matrix(r_c, r_f, dr, dz, n_r, n_z);
42 fprintf(' Matrix size: %d x %d, nnz = %d\\n', size(A_poisson,1), size(
43 A_poisson,2), nnz(A_poisson));
44 poisson_solver = decomposition(A_poisson, 'lu');
45
46 %% History arrays
47 hist_cont = zeros(n_steps, 1);
48 hist_vel = zeros(n_steps, 1);
49 hist_temp = zeros(n_steps, 1);
50
51 %% Time loop
52 fprintf('\\nStarting time integration ...\\n');
53 converged_step = 0;
54 rhs_u_old = [];
55 rhs_w_old = [];

```

```

53 rhs_T_old = [];
54
55 for step = 1:n_steps
56
57     % AB2 predictor + one incremental pressure correction
58     [u_new, w_new, p_new, T_new, rhs_u, rhs_w, rhs_T] = ...
59         ab2_step(u, w, p, T, rhs_u_old, rhs_w_old, rhs_T_old, ...
60             r_c, r_f, dr, dz, dt, Re, Pr, n_r, n_z, ...
61             poisson_solver, alpha_p);
62
63     % Convergence check
64     res = compute_residuals(u_new, u, w_new, w, T_new, T, r_c, r_f, dr,
65         dz, dt);
66     hist_cont(step) = res.continuity;
67     hist_vel(step) = res.momentum;
68     hist_temp(step) = res.temperature;
69
70     % Advance
71     u = u_new; w = w_new; p = p_new; T = T_new;
72     rhs_u_old = rhs_u;
73     rhs_w_old = rhs_w;
74     rhs_T_old = rhs_T;
75
76     % Progress print
77     if mod(step, c.output_interval) == 0
78         fprintf(' step %6d/%d R_cont=%.2e R_vel=%.2e R_T=%.2e\n',
79             ...
80                 step, n_steps, res.continuity, res.momentum, res.
81                 temperature);
82         end
83
84         if check_convergence(res, c.tol_continuity, c.tol_velocity, c.
85             tol_temperature)
86             fprintf('\n Converged at step %d!\n', step);
87             converged_step = step;
88             break;
89         end
90     end
91
92     if converged_step == 0
93         converged_step = n_steps;
94         warning('Maximum step count reached before all convergence
95             tolerances were met.');
```

```

105 if ~exist(plots_dir, 'dir')
106     mkdir(plots_dir);
107 end
108
109 %% Plot 1: axial velocity contour
110 figure('Visible', 'off');
111 contourf(z_c, r_c, w_cc, 20, 'LineStyle', 'none');
112 hold on;
113 % Display a decimated quiver layer so individual arrows remain readable.
114 % The underlying contour still contains every cell; vectors use every 5
    th
115 % axial and every 2nd radial cell centre.
116 z_arrow = 1:5:length(z_c);
117 r_arrow = 1:2:length(r_c);
118 q = quiver(z_c(z_arrow), r_c(r_arrow), w_cc(r_arrow, z_arrow), ...
119           u_cc(r_arrow, z_arrow), 0.45, 'k', 'LineWidth', 0.5);
120 q.Clipping = 'on';
121 q.MaxHeadSize = 5.0;
122 xlim([0, 50]);
123 ylim([0, 0.5]);
124 colorbar; xlabel('z/D'); ylabel('r/D');
125 title('Axial velocity w/W_{in} and velocity vectors');
126 saveas(gcf, fullfile(plots_dir, 'w_contour.png'));
127 fprintf(' Saved w_contour.png\n');
128
129 %% Plot 2: radial velocity contour
130 % u is ~0 over most of the domain (flow is fully developed for z* > ~5,
131 % i.e. 90% of the pipe length) except for a strong, spatially localised
132 % entrance-corner value near (r=0.5, z=0). A linear color scale spanning
133 % the raw min/max is dominated by that single feature and renders the
134 % entire developed region as one flat color. Instead use a diverging
135 % colormap centred at zero with a robust (98th-percentile) symmetric
136 % limit, so the near-zero bulk keeps visible contrast and the entrance
137 % feature simply saturates the color scale rather than washing it out.
138 figure('Visible', 'off');
139 u_abs_sorted = sort(abs(u_cc(:)));
140 u_lim = u_abs_sorted(max(1, round(0.98 * numel(u_abs_sorted))));
141 if u_lim <= 0
142     u_lim = max(abs(u_cc(:)));
143 end
144 if u_lim <= 0
145     u_lim = 1;
146 end
147 contourf(z_c, r_c, u_cc, linspace(-u_lim, u_lim, 21), 'LineStyle', 'none
    ');
148 clim([-u_lim, u_lim]);
149 colorbar; colormap(gca, diverging_bwr(256)); xlabel('z/D'); ylabel('r/D'
    );
150 title('Radial velocity u/W_{in}');
151 saveas(gcf, fullfile(plots_dir, 'u_contour.png'));
152 fprintf(' Saved u_contour.png\n');
153
154 %% Plot 3: temperature contour
155 figure('Visible', 'off');
156 contourf(z_c, r_c, T, 20, 'LineStyle', 'none');
157 colorbar; colormap(gca, 'hot'); xlabel('z/D'); ylabel('r/D');
158 title('Non-dimensional temperature \theta');
159 saveas(gcf, fullfile(plots_dir, 'theta_contour.png'));

```

```

160 fprintf('  Saved theta_contour.png\n');
161
162 %% Plot 4: velocity profiles at selected z-positions
163 z_plot = [5, 10, 20, 30, 40, 50];
164 figure('Visible', 'off'); hold on;
165 w_an = analytical_velocity(r_c);
166 for k = 1:length(z_plot)
167     [~, jj] = min(abs(z_c - z_plot(k)));
168     plot(r_c, w_cc(:, jj), '-o', 'MarkerSize', 3, 'DisplayName', sprintf(
        'z*=%.0f', z_plot(k)));
169 end
170 plot(r_c, w_an, 'k--', 'LineWidth', 1.5, 'DisplayName', 'Analytical');
171 xlabel('r/D'); ylabel('w/W_{in}');
172 title('Velocity profiles at selected z-positions');
173 legend('Location', 'best'); grid on;
174 saveas(gcf, fullfile(plots_dir, 'w_profiles.png'));
175 fprintf('  Saved w_profiles.png\n');
176
177 %% Plot 5: temperature profiles
178 figure('Visible', 'off'); hold on;
179 for k = 1:length(z_plot)
180     [~, jj] = min(abs(z_c - z_plot(k)));
181     plot(r_c, T(:, jj), '-o', 'MarkerSize', 3, 'DisplayName', sprintf('z
        *=%.0f', z_plot(k)));
182 end
183 % Analytical fully-developed profile at z*=50 (approved form; see
184 % Report/nusselt_number_analysis.md)
185 [~, j_Tfd] = min(abs(z_c - z_plot(end)));
186 z_Tfd = z_c(j_Tfd);
187 theta_an = analytical_temperature(r_c, z_Tfd, Re, Pr);
188 plot(r_c, theta_an, 'k--', 'LineWidth', 1.5, ...
189     'DisplayName', sprintf('Analytical profile at z*=%.1f', z_Tfd));
190 xlabel('r/D'); ylabel('\theta');
191 title('Temperature profiles at selected z-positions');
192 legend('Location', 'best'); grid on;
193 saveas(gcf, fullfile(plots_dir, 'T_profiles.png'));
194 fprintf('  Saved T_profiles.png\n');
195
196 %% Plot 6: convergence history
197 figure('Visible', 'off');
198 semilogy(1:converged_step, hist_cont, 'b-', 'DisplayName', 'Continuity')
199 ;
200 hold on;
201 semilogy(1:converged_step, hist_vel, 'r-', 'DisplayName', 'Momentum');
202 semilogy(1:converged_step, hist_temp, 'g-', 'DisplayName', 'Temperature'
203 );
204 xlabel('Time step'); ylabel('Residual');
205 title('Convergence history');
206 legend('Location', 'best'); grid on;
207 saveas(gcf, fullfile(plots_dir, 'convergence.png'));
208 fprintf('  Saved convergence.png\n');
209
210 %% Plot 7: comparison with analytical at z*=50
211 [~, j_fd] = min(abs(z_c - 50.0));
212 w_num = w_cc(:, j_fd);
213 w_ana = analytical_velocity(r_c);
214 figure('Visible', 'off');

```

```

214 subplot(2,1,1);
215 plot(r_c, w_num, 'bo-', 'MarkerSize', 4, 'DisplayName', 'Numerical');
216 hold on;
217 plot(r_c, w_ana, 'r--', 'LineWidth', 1.5, 'DisplayName', 'Analytical');
218 xlabel('r/D'); ylabel('w/W_{in}');
219 title('Velocity at z*=50: Numerical vs Analytical');
220 legend('Location', 'best'); grid on;
221
222 subplot(2,1,2);
223 plot(r_c, abs(w_num - w_ana), 'k-', 'LineWidth', 1);
224 xlabel('r/D'); ylabel('|Error|');
225 title('Absolute error'); grid on;
226 saveas(gcf, fullfile(plots_dir, 'w_comparison.png'));
227 fprintf(' Saved w_comparison.png\n');
228
229 fprintf('\nAll plots saved to %s\n', plots_dir);
230
231 %% Save final fields for offline inspection / diagnostics
232 save(fullfile(plots_dir, '..', 'src_matlab', 'last_run.mat'), ...
233      'u', 'w', 'p', 'T', 'u_cc', 'w_cc', 'r_c', 'z_c', 'hist_cont', '
      hist_vel', 'hist_temp');
234
235 close all;

```

Listing 3: main.m — Main simulation loop and plotting

B.3 Grid Generation

```

1 function [r_c, r_f, z_c, z_f, dr, dz] = make_grid(n_r, n_z, L_over_D)
2 % MAKE_GRID Generate staggered grid for axisymmetric pipe flow.
3 %
4 % [r_c, r_f, z_c, z_f, dr, dz] = make_grid(n_r, n_z, L_over_D)
5 %
6 % Outputs (all non-dimensional, scaled by D):
7 % r_c : (n_r x 1) cell-centre radial coords [0 < r < 0.5]
8 % r_f : (n_r+1 x 1) r-face coords [0 <= r <= 0.5]
9 % z_c : (n_z x 1) cell-centre axial coords
10 % z_f : (n_z+1 x 1) z-face coords
11 % dr : radial spacing
12 % dz : axial spacing
13
14 dr = 0.5 / n_r;
15 dz = L_over_D / n_z;
16
17 r_f = linspace(0, 0.5, n_r + 1)'; % column vector
18 r_c = 0.5 * (r_f(1:end-1) + r_f(2:end));
19
20 z_f = linspace(0, L_over_D, n_z + 1)';
21 z_c = 0.5 * (z_f(1:end-1) + z_f(2:end));
22 end

```

Listing 4: make_grid.m — Staggered grid generation

B.4 Boundary Conditions

```

1 function [u, w, T] = apply_all_bc(u, w, T)
2 % APPLY_ALL_BC Apply all boundary conditions in one call.
3
4     u = apply_bc_u(u);
5     w = apply_bc_w(w);
6     T = apply_bc_T(T);
7 end

```

Listing 5: apply_all_bc.m — Driver applying all field boundary conditions

```

1 function w = apply_bc_w(w)
2 % APPLY_BC_W Enforce axial velocity boundary conditions.
3 %
4 %     w(:, 1)    = 1    inlet: uniform w* = 1
5 %     w(:, end) = w(:, end-1) outlet: zero-gradient
6
7     w(:, 1)    = 1.0;           % inlet
8     w(:, end) = w(:, end-1);   % outlet zero-gradient
9 end

```

Listing 6: apply_bc_w.m — Axial velocity boundary conditions

```

1 function u = apply_bc_u(u)
2 % APPLY_BC_U Enforce radial velocity boundary conditions.
3 %
4 %     u(1, :)    = 0    centreline (r = 0, symmetry)
5 %     u(end, :) = 0    wall (r = R, no penetration)
6 %
7 %     NOTE: u(:,1) is at z_c(1), NOT the inlet face.
8 %     The inlet BC u=0 at z=0 is handled by the ghost cell in the
9 %     predictor.
10
11     u(1, :)    = 0;    % axis
12     u(end, :)  = 0;    % wall
13 end

```

Listing 7: apply_bc_u.m — Radial velocity boundary conditions

```

1 function T = apply_bc_T(T)
2 % APPLY_BC_T No-op for T -- kept only so apply_all_bc has a uniform
3 % interface across u, w, T.
4 %
5 %     Inlet (Dirichlet theta=0) and outlet (zero-gradient) are enforced
6 %     purely through ghost cells inside compute_rhs_T (T_ghost_in=-T(:,1),
7 %     T_ghost_out=T(:,end)) -- NOT by overwriting the T(:,1)/T(:,end) cell
8 %     values here. Those are cell centres a half-cell away from the actual
9 %     inlet/outlet faces, so clamping them directly would over-constrain
10 %     the PDE and is inconsistent with the ghost-cell treatment used
11 %     everywhere else (see compute_rhs_T.m and
12 %     Report/nusselt_number_analysis.md for the wall BC derivation).
13 %
14 %     Wall heat flux BC (dtheta/dr = +Re*Pr) is likewise handled entirely
15 %     via ghost cells inside compute_rhs_T.m.
16 end

```

Listing 8: apply_bc_T.m — Temperature boundary conditions

B.5 Momentum Solver

```
1 function w_star = predictor_w(w, u, r_c, r_f, dr, dz, dt, Re)
2 % PREDICTOR_W Advance axial velocity by one explicit Euler step (no
3 % pressure). Thin wrapper around compute_rhs_w; the main time loop
  uses
4 % compute_rhs_w directly (via ab2_step) to build the AB2 update.
5 %
6 % Returns w_star : (n_r, n_z+1) predicted axial velocity (BCs NOT yet
  reapplied)
7
8 rhs = compute_rhs_w(w, u, r_c, r_f, dr, dz, Re);
9 n_z1 = size(w, 2);
10 n_z = n_z1 - 1;
11 w_star = w;
12 w_star(:, 2:n_z) = w(:, 2:n_z) + dt * rhs;
13 end
```

Listing 9: predictor_w.m — Axial velocity predictor

```
1 function u_star = predictor_u(u, w, r_c, r_f, dr, dz, dt, Re)
2 % PREDICTOR_U Advance radial velocity by one explicit Euler step (no
3 % pressure). Thin wrapper around compute_rhs_u; the main time loop
  uses
4 % compute_rhs_u directly (via ab2_step) to build the AB2 update.
5 %
6 % Returns u_star : (n_r+1, n_z)
7
8 rhs = compute_rhs_u(u, w, r_c, r_f, dr, dz, Re);
9 u_star = u;
10 u_star(2:end-1, :) = u(2:end-1, :) + dt * rhs;
11 end
```

Listing 10: predictor_u.m — Radial velocity predictor

```
1 function face_val = upwind2_face(phi_im1, phi_i, phi_ip1, phi_ip2, vel)
2 % UPWIND2_FACE 2nd-order upwind (linear-extrapolation / LUD) face value
  .
3 %
4 % face value = 1.5*phi_i - 0.5*phi_im1 if vel > 0 (upwind: low-
  index side)
5 %               = 1.5*phi_ip1 - 0.5*phi_ip2 if vel < 0 (upwind: high-
  index side)
6 %
7 % phi_im1, phi_i, phi_ip1, phi_ip2 : values at relative positions i-1,
  i,
8 % i+1, i+2 with respect to the face sitting between cells i and i
  +1.
9 % vel : transport velocity at the face.
10 %
11 % Retains the upwind stability bias needed at high cell Peclet number
12 % while being formally 2nd-order accurate (unlike pure central
13 % differencing or 1st-order upwind).
14
15 face_val = (1.5 * phi_i - 0.5 * phi_im1) .* (vel > 0) ...
16           + (1.5 * phi_ip1 - 0.5 * phi_ip2) .* (vel <= 0);
17 end
```

Listing 11: upwind2_face.m — 2nd-order upwind (linear-extrapolation) face-value helper

```

1 function rhs = compute_rhs_w(w, u, r_c, r_f, dr, dz, Re)
2 % COMPUTE_RHS_W Explicit RHS of the axial-momentum equation (no
   pressure,
3 % no time discretization): rhs = -conv_r - conv_z + visc_r + visc_z
4 %
5 % Kept separate from time-stepping for the AB2 history update.
6 %
7 % Convective fluxes use a 2nd-order upwind (linear-extrapolation / LUD
   )
8 % scheme -- see upwind2_face.m -- retaining the upwind stability bias
   at
9 % high cell Peclet number while being formally 2nd-order accurate,
10 % matching the diffusive terms (2nd-order central differencing).
11 %
12 % w : (n_r, n_z+1) current axial velocity (BCs already applied)
13 % u : (n_r+1, n_z) current radial velocity (BCs already applied)
14 % r_c, r_f, dr, dz, Re : as usual
15 %
16 % Returns rhs : (n_r, n_z-1) interior-face RHS (excludes j=1 and j=
   n_z+1)
17
18 [n_r, n_z1] = size(w);
19 n_z = n_z1 - 1;
20
21 % ---- ghost extension in r for w (depth 1, used by diffusion)
   -----
22 w_ghost_wall = -w(end, :);
23 w_ghost_axis = w(1, :);
24 w_ext = [w_ghost_axis; w; w_ghost_wall]; % (n_r+2, n_z+1)
25
26 % ---- second ghost layer in r, needed by the 2nd-order-upwind
   stencil --
27 w_ghost_axis2 = w(2, :);
28 w_ghost_wall2 = -w(end-1, :);
29 w_ext2 = [w_ghost_axis2; w_ghost_axis; w; w_ghost_wall;
   w_ghost_wall2]; % (n_r+4, n_z+1)
30 % w_ext2(i+2, :) = w(i, :)
31
32 % ---- interior slice j=2..n_z
   -----
33 w_int = w(:, 2:n_z); % (n_r, n_z-1)
34
35 % ---- u interpolated to z_f(j) for j=2..n_z
   -----
36 u_at_zf = 0.5 * (u(:, 1:end-1) + u(:, 2:end)); % (n_r+1, n_z-1)
37
38 % --- Radial convective flux (1/r) d(r*u*w)/dr ---
39 w_r_im2 = w_ext2(1:n_r, 2:n_z);
40 w_r_im1 = w_ext2(2:n_r+1, 2:n_z);
41 w_r_i = w_ext2(3:n_r+2, 2:n_z); % = w_int
42 w_r_ip1 = w_ext2(4:n_r+3, 2:n_z);
43 w_r_ip2 = w_ext2(5:n_r+4, 2:n_z);
44
45 u_E = u_at_zf(2:end, :);
46 w_E_upwind = upwind2_face(w_r_im1, w_r_i, w_r_ip1, w_r_ip2, u_E);
47
48 u_I = u_at_zf(1:end-1, :);

```

```

49     w_I_upwind = upwind2_face(w_r_im2, w_r_im1, w_r_i, w_r_ip1, u_I);
50
51     % --- Axial convective flux  $d(w^2)/dz$  ---
52     % w already has real values exactly at the inlet/outlet faces, so
53     % only
54     % one extra ghost point beyond each end is needed (constant
55     % extension).
56     w_zghost_before = w(:, 1);
57     w_zghost_after  = w(:, end);
58     w_zext1 = [w_zghost_before, w, w_zghost_after];    % (n_r, n_z+3)
59
60     w_z_jm2 = w_zext1(:, 1:n_z-1);
61     w_z_jm1 = w_zext1(:, 2:n_z);
62     w_z_j    = w_zext1(:, 3:n_z+1);    % = w_int
63     w_z_jp1 = w_zext1(:, 4:n_z+2);
64     w_z_jp2 = w_zext1(:, 5:n_z+3);
65
66     w_N = 0.5 * (w_z_j + w_z_jp1);
67     w_N_upwind = upwind2_face(w_z_jm1, w_z_j, w_z_jp1, w_z_jp2, w_N);
68
69     w_S = 0.5 * (w_z_jm1 + w_z_j);
70     w_S_upwind = upwind2_face(w_z_jm2, w_z_jm1, w_z_j, w_z_jp1, w_S);
71
72     % ---- reshaping for broadcasting
73     -----
74     rc = r_c;
75     rf_o = r_f(2:end);
76     rf_i = r_f(1:end-1);
77
78     % ---- convective fluxes (2nd-order upwind)
79     -----
80     conv_r = (rf_o .* u_E .* w_E_upwind - rf_i .* u_I .* w_I_upwind) ./
81     (rc .* dr);
82     conv_z = (w_N .* w_N_upwind - w_S .* w_S_upwind) / dz;
83
84     % ---- diffusive fluxes (central, 2nd order)
85     -----
86     visc_r = (rf_o .* (w_ext(3:end, 2:n_z) - w_int) ...
87             - rf_i .* (w_int - w_ext(1:end-2, 2:n_z))) ./ (rc * dr^2) /
88     Re;
89
90     visc_z = (w(:, 3:n_z+1) - 2*w_int + w(:, 1:n_z-1)) / dz^2 / Re;
91
92     rhs = -conv_r - conv_z + visc_r + visc_z;
93 end

```

Listing 12: compute_rhs_w.m — Axial momentum RHS: 2nd-order upwind convection, central diffusion

```

1 function rhs = compute_rhs_u(u, w, r_c, r_f, dr, dz, Re)
2 % COMPUTE_RHS_U Explicit RHS of the radial-momentum equation (no
3 % pressure,
4 % no time discretization). Reused by the AB2 time integrator.
5 % Convective fluxes use a 2nd-order upwind (linear-extrapolation / LUD
6 % scheme -- see upwind2_face.m.
7 %

```

```

8 % u : (n_r+1, n_z)
9 % w : (n_r, n_z+1)
10 % r_c, r_f, dr, dz, Re : as usual
11 %
12 % Returns rhs : (n_r-1, n_z) interior-face RHS (excludes i=1 and i=
n_r+1)
13
14 [n_rpl, n_z] = size(u);
15 n_r = n_rpl - 1;
16
17 % ---- ghost extension in z for u (depth 2), needed by diffusion and
by --
18 % the 2nd-order-upwind stencil.
19 u_ghost_in1 = -u(:, 1);
20 u_ghost_in2 = -u(:, 2);
21 u_ghost_out1 = u(:, end);
22 u_ghost_out2 = u(:, end);
23 u_ext = [u_ghost_in1, u, u_ghost_out1]; % (
n_r+1, n_z+2), depth 1 (diffusion)
24 u_zext2 = [u_ghost_in2, u_ghost_in1, u, u_ghost_out1, u_ghost_out2];
% (n_r+1, n_z+4)
25
26 % ---- second ghost layer in r, needed by the 2nd-order-upwind
stencil ---
27 u_ghost_axis_ext = -u(2, :); % represents "u(0)" (one before
axis)
28 u_ghost_wall_ext = -u(end-1, :); % represents "u(n_r+2)" (one
beyond wall)
29 u_ext_r = [u_ghost_axis_ext; u; u_ghost_wall_ext]; % (n_r+3, n_z)
30
31 % ---- interior slice i=2..n_r
-----
32 u_int = u(2:end-1, :); % (n_r-1, n_z)
33
34 % --- Radial convective flux (1/r) d(r*u^2)/dr ---
35 u_r_im2 = u_ext_r(1:n_r-1, :);
36 u_r_im1 = u_ext_r(2:n_r, :);
37 u_r_i = u_ext_r(3:n_r+1, :); % = u_int
38 u_r_ip1 = u_ext_r(4:n_r+2, :);
39 u_r_ip2 = u_ext_r(5:n_r+3, :);
40
41 u_out = 0.5 * (u_int + u(3:end, :));
42 u_in = 0.5 * (u(1:end-2, :) + u_int);
43
44 u_out_upwind = upwind2_face(u_r_im1, u_r_i, u_r_ip1, u_r_ip2, u_out)
;
45 u_in_upwind = upwind2_face(u_r_im2, u_r_im1, u_r_i, u_r_ip1, u_in);
46
47 % ---- face velocities for axial convection
-----
48 w_at_rf = 0.5 * (w(1:end-1, :) + w(2:end, :)); % (n_r-1, n_z+1)
49 w_top = w_at_rf(:, 2:end);
50 w_bot = w_at_rf(:, 1:end-1);
51
52 % --- Axial convective flux d(u*w)/dz --- (2nd-order upwind,
interior rows)
53 u_z_jm2 = u_zext2(2:end-1, 1:n_z);
54 u_z_jm1 = u_zext2(2:end-1, 2:n_z+1);

```

```

55     u_z_j      = u_zext2(2:end-1, 3:n_z+2);    % = u_int
56     u_z_jp1    = u_zext2(2:end-1, 4:n_z+3);
57     u_z_jp2    = u_zext2(2:end-1, 5:n_z+4);
58
59     u_top_upwind = upwind2_face(u_z_jm1, u_z_j, u_z_jp1, u_z_jp2, w_top)
60 ;
61     u_bot_upwind = upwind2_face(u_z_jm2, u_z_jm1, u_z_j, u_z_jp1, w_bot)
62 ;
63
64     % ---- reshaping for broadcasting
65     -----
66     rc_o      = r_c(2:end);
67     rc_i      = r_c(1:end-1);
68     rf_int    = r_f(2:end-1);
69
70     % ---- convective fluxes (2nd-order upwind)
71     -----
72     conv_r = (rc_o .* u_out .* u_out_upwind - rc_i .* u_in .*
73 u_in_upwind) ./ (rf_int .* dr);
74     conv_z = (w_top .* u_top_upwind - w_bot .* u_bot_upwind) / dz;
75
76     % ---- diffusive fluxes (central, 2nd order)
77     -----
78     visc_r = ((rc_o .* (u(3:end, :) - u_int) - rc_i .* (u_int - u(1:end
79 -2, :))) ./ (rf_int * dr^2) ...
80             - u_int ./ rf_int.^2) / Re;
81
82     visc_z = (u_ext(2:end-1, 3:end) - 2*u_int + u_ext(2:end-1, 1:end-2))
83 / dz^2 / Re;
84
85     rhs = -conv_r - conv_z + visc_r + visc_z;
86 end

```

Listing 13: compute_rhs_u.m — Radial momentum RHS: 2nd-order upwind convection, central diffusion

```

1 function [u_new, w_new, p_new, T_new, rhs_u, rhs_w, rhs_T] = ...
2     ab2_step(u, w, p, T, rhs_u_old, rhs_w_old, rhs_T_old, ...
3             r_c, r_f, dr, dz, dt, Re, Pr, n_r, n_z, A_poisson, alpha_p)
4 % AB2_STEP Adams-Bashforth 2 with one incremental pressure correction.
5 % The first call uses forward Euler when the previous RHS arrays are
6 % empty.
7
8     rhs_u = compute_rhs_u(u, w, r_c, r_f, dr, dz, Re);
9     rhs_w = compute_rhs_w(w, u, r_c, r_f, dr, dz, Re);
10    rhs_T = compute_rhs_T(T, u, w, r_c, r_f, dr, dz, Re, Pr);
11
12    if isempty(rhs_u_old)
13        explicit_u = rhs_u;
14        explicit_w = rhs_w;
15        explicit_T = rhs_T;
16    else
17        explicit_u = 1.5 * rhs_u - 0.5 * rhs_u_old;
18        explicit_w = 1.5 * rhs_w - 0.5 * rhs_w_old;
19        explicit_T = 1.5 * rhs_T - 0.5 * rhs_T_old;
20    end
21
22    % The old pressure gradient makes the Poisson unknown a correction p

```

```

22     u_star = u;
23     grad_p_r = (p(2:end, :) - p(1:end-1, :)) / dr;
24     u_star(2:end-1, :) = u(2:end-1, :) ...
25         + dt * (explicit_u - grad_p_r);
26
27     w_star = w;
28     grad_p_z = (p(:, 2:end) - p(:, 1:end-1)) / dz;
29     w_star(:, 2:end-1) = w(:, 2:end-1) ...
30         + dt * (explicit_w - grad_p_z);
31     u_star = apply_bc_u(u_star);
32     w_star = apply_bc_w(w_star);
33
34     p_prime = solve_poisson(A_poisson, u_star, w_star, ...
35                             r_c, r_f, dr, dz, dt, n_r, n_z);
36     [u_new, w_new] = correct_velocity(u_star, w_star, ...
37                                     p_prime, dr, dz, dt);
38     u_new = apply_bc_u(u_new);
39     w_new = apply_bc_w(w_new);
40
41     p_new = update_pressure(p, p_prime, alpha_p);
42     T_new = T + dt * explicit_T;
43 end

```

Listing 14: ab2_step.m — AB2 step with one incremental pressure correction

B.6 Poisson Solver and Pressure Correction

```

1 function A = build_poisson_matrix(r_c, r_f, dr, dz, n_r, n_z)
2 % BUILD_POISSON_MATRIX Assemble sparse Poisson matrix (called once).
3 %
4 % A = build_poisson_matrix(r_c, r_f, dr, dz, n_r, n_z)
5 %
6 % Returns A : sparse matrix (n_r*n_z x n_r*n_z)
7 %
8 % Flat index: k = (i-1)*n_z + j (1-based)
9 %
10 % BCs for p':
11 %   Centreline (i=1):      Neumann (r_f(1)=0, flux vanishes naturally)
12 %   Wall (i=n_r):         Neumann (outer flux = 0)
13 %   Inlet (j=1):          Neumann (south flux = 0)
14 %   Outlet (j=n_z):       Dirichlet p' = 0
15
16 N = n_r * n_z;
17
18 % Pre-allocate triplets (at most 5 entries per row)
19 rows = zeros(5*N, 1);
20 cols = zeros(5*N, 1);
21 vals = zeros(5*N, 1);
22 idx = 0;
23
24 for i = 1:n_r
25     for j = 1:n_z
26         k = (i-1)*n_z + j;
27
28         % Dirichlet at outlet (j = n_z): p' = 0
29         if j == n_z

```

```

30         idx = idx + 1;
31         rows(idx) = k; cols(idx) = k; vals(idx) = 1.0;
32         continue;
33     end
34
35     % Coefficients from cylindrical Poisson
36     rf_o = r_f(i + 1); %  $r_{i+1/2}$ 
37     rf_i_val = r_f(i); %  $r_{i-1/2}$ 
38     rc = r_c(i);
39
40     coeff_E = rf_o / (rc * dr^2); %  $p'(i+1, j)$ 
41     coeff_W = rf_i_val / (rc * dr^2); %  $p'(i-1, j)$ 
42     coeff_N = 1.0 / dz^2; %  $p'(i, j+1)$ 
43     coeff_S = 1.0 / dz^2; %  $p'(i, j-1)$ 
44
45     % Neumann wall (i = n_r): outer flux = 0
46     if i == n_r
47         coeff_E = 0.0;
48     end
49
50     % Neumann inlet (j = 1): south flux = 0
51     if j == 1
52         coeff_S = 0.0;
53     end
54
55     % Centre coefficient
56     coeff_C = -(coeff_E + coeff_W + coeff_N + coeff_S);
57
58     % Diagonal
59     idx = idx + 1;
60     rows(idx) = k; cols(idx) = k; vals(idx) = coeff_C;
61
62     % East (i+1, j)
63     if i < n_r
64         idx = idx + 1;
65         rows(idx) = k; cols(idx) = i*n_z + j; vals(idx) =
coeff_E;
66     end
67
68     % West (i-1, j)
69     if i > 1
70         idx = idx + 1;
71         rows(idx) = k; cols(idx) = (i-2)*n_z + j; vals(idx) =
coeff_W;
72     end
73
74     % North (i, j+1)
75     if j < n_z
76         idx = idx + 1;
77         rows(idx) = k; cols(idx) = (i-1)*n_z + (j+1); vals(idx)
= coeff_N;
78     end
79
80     % South (i, j-1)
81     if j > 1
82         idx = idx + 1;
83         rows(idx) = k; cols(idx) = (i-1)*n_z + (j-1); vals(idx)
= coeff_S;

```

```

84         end
85     end
86 end
87
88 % Trim and build sparse matrix
89 rows = rows(1:idx);
90 cols = cols(1:idx);
91 vals = vals(1:idx);
92 A = sparse(rows, cols, vals, N, N);
93 end

```

Listing 15: build_poisson_matrix.m — Sparse Poisson matrix assembly

```

1 function p_prime = solve_poisson(A, u_star, w_star, r_c, r_f, dr, dz, dt
  , n_r, n_z)
2 % SOLVE_POISSON Solve  $A * p' = b$  for pressure correction.
3 %
4 % p_prime = solve_poisson(A, u_star, w_star, r_c, r_f, dr, dz, dt, n_r
  , n_z)
5 %
6 % RHS:  $b(i, j) = (1/dt) * \text{div}(u^*)$ 
7 % Returns p_prime : (n_r, n_z)
8
9 rf_o = r_f(2:end); % (n_r, 1)
10 rf_i = r_f(1:end-1); % (n_r, 1)
11 rc = r_c; % (n_r, 1)
12
13 % Radial divergence
14 div_u = (rf_o .* u_star(2:end, :) - rf_i .* u_star(1:end-1, :)) ./ (
rc * dr);
15
16 % Axial divergence
17 div_w = (w_star(:, 2:end) - w_star(:, 1:end-1)) / dz;
18
19 b = (div_u + div_w) / dt; % (n_r, n_z)
20
21 % Outlet row (j=n_z) has Dirichlet  $p'=0 \rightarrow \text{RHS} = 0$ 
22 b(:, end) = 0.0;
23
24 % Flatten and solve
25 b_flat = b(:); % column-major flatten (same as reshape(
b, [], 1))
26 % MATLAB sparse direct solve (backslash)
27 % Note: MATLAB stores column-major, Python row-major. The flat index
28 % must match the matrix assembly order:  $k = (i-1)*n_z + j$ .
29 % Reshaping b column-major: b(:) goes column by column (j varies
fastest
30 % for fixed i when b is (n_r, n_z)). But our matrix uses  $k=(i-1)*n_z
+j$ ,
31 % where j varies fastest for fixed i. This matches b(:) since MATLAB
32 % flattens column-major on (n_r, n_z)  $\rightarrow$  i varies fastest.
33 % We need to transpose to match: b' is (n_z, n_r), then b'(:) has
34 % j varying slowest... no. Let me be precise.
35 %
36 % Matrix assembly:  $k = (i-1)*n_z + j$ , so for  $i=1$ ,  $j=1..n_z \rightarrow k=1..
n_z$ .
37 % MATLAB b(:) for b of size (n_r, n_z) gives b(1,1), b(2,1), ..., b(
n_r,1),

```

```

38 % b(1,2), ..., i.e. i varies fastest. But we need j to vary
    fastest.
39 % Solution: transpose b before flattening.
40 b_flat = reshape(b', [], 1); % b' is (n_z, n_r), flatten -> j
    varies fastest
41
42 p_prime_flat = A \ b_flat;
43
44 % Reshape back: p_prime_flat has k=(i-1)*n_z+j ordering
45 p_prime = reshape(p_prime_flat, n_z, n_r)'; % (n_r, n_z)
46 end

```

Listing 16: solve_poisson.m — Poisson solve using MATLAB backslash

```

1 function div = compute_divergence(u, w, r_c, r_f, dr, dz)
2 % COMPUTE_DIVERGENCE Pointwise discrete divergence of velocity field.
3 %
4 % div(i,j) = [r_{i+1/2}*u(i+1,j) - r_{i-1/2}*u(i,j)] / (r_c(i)*dr)
5 %             + [w(i,j+1) - w(i,j)] / dz
6 %
7 % Returns div : (n_r, n_z)
8
9 rf_o = r_f(2:end); % (n_r, 1)
10 rf_i = r_f(1:end-1); % (n_r, 1)
11 rc = r_c; % (n_r, 1)
12
13 div_u = (rf_o .* u(2:end, :) - rf_i .* u(1:end-1, :)) ./ (rc * dr);
14 div_w = (w(:, 2:end) - w(:, 1:end-1)) / dz;
15
16 div = div_u + div_w;
17 end

```

Listing 17: compute_divergence.m — Discrete velocity divergence (Poisson RHS and continuity residual)

```

1 function [u_new, w_new] = correct_velocity(u_star, w_star, p_prime, dr,
    dz, dt)
2 % CORRECT_VELOCITY Apply pressure correction to obtain div-free
    velocities.
3 %
4 % u^{n+1}(i,j) = u*(i,j) - dt * (p'(i,j) - p'(i-1,j)) / dr
5 % w^{n+1}(i,j) = w*(i,j) - dt * (p'(i,j) - p'(i,j-1)) / dz
6
7 u_new = u_star;
8 w_new = w_star;
9
10 % Radial velocity at interior r-faces (rows 2:end-1)
11 u_new(2:end-1, :) = u_star(2:end-1, :) ...
12     - dt * (p_prime(2:end, :) - p_prime(1:end-1, :)) / dr;
13
14 % Axial velocity at interior z-faces (columns 2:end-1)
15 w_new(:, 2:end-1) = w_star(:, 2:end-1) ...
16     - dt * (p_prime(:, 2:end) - p_prime(:, 1:end-1)) / dz;
17 end

```

Listing 18: correct_velocity.m — Velocity correction step

```

1 function p_new = update_pressure(p, p_prime, zeta)

```

```

2 % UPDATE_PRESSURE Accumulate relaxed correction: p_new = p + zeta*
   p_prime.
3
4     p_new = p + zeta * p_prime;
5 end

```

Listing 19: update_pressure.m — Additive pressure-correction update

B.7 Temperature Solver

```

1 function rhs = compute_rhs_T(T, u, w, r_c, r_f, dr, dz, Re, Pr)
2 % COMPUTE_RHS_T Explicit RHS of the energy equation (no time
3 % discretization): rhs = -conv_r - conv_z + visc_r + visc_z
4 %
5 % Reused as the current and stored previous RHS in ab2_step.m.
6 %
7 % Convective fluxes use a 2nd-order upwind (linear-extrapolation / LUD
   )
8 % scheme -- see upwind2_face.m. Diffusive terms use standard 2nd-order
9 % central differencing.
10 %
11 % Wall BC: dtheta/dr* = +Re*Pr (see Report/nusselt_number_analysis.md
12 % for the derivation -- NOT -1, which drops the Re*Pr factor from the
13 % non-dimensionalization).
14 % Implemented as a ghost cell: theta_ghost(n_r+1,:) = theta(n_r,:) +
   dr*Re*Pr
15 %
16 % T : (n_r, n_z) current theta (BCs already applied)
17 % u : (n_r+1, n_z) radial velocity at r-faces
18 % w : (n_r, n_z+1) axial velocity at z-faces
19 % r_c, r_f, dr, dz, Re, Pr : as usual
20 %
21 % Returns rhs : (n_r, n_z)
22
23 [n_r, n_z] = size(T);
24 alpha = 1.0 / (Re * Pr);
25 RePr = Re * Pr;
26
27 % ---- ghost extension in r (depth 1, used by diffusion)
   -----
28 T_ghost_axis = T(1, :);
29 T_ghost_wall = T(end, :) + dr * RePr;
30 T_ext = [T_ghost_axis; T; T_ghost_wall]; % (n_r+2, n_z)
31
32 % ---- ghost extension in z (depth 1, used by diffusion)
   -----
33 T_ghost_in = -T(:, 1);
34 T_ghost_out = T(:, end);
35 T_zext = [T_ghost_in, T, T_ghost_out]; % (n_r, n_z+2)
36
37 % ---- second ghost layer, needed by the 2nd-order-upwind stencil
   -----
38 T_ghost_axis2 = T(2, :);
39 T_ghost_wall2 = T(end, :) + 2.0 * dr * RePr;
40 T_ext2 = [T_ghost_axis2; T_ghost_axis; T; T_ghost_wall;
   T_ghost_wall2]; % (n_r+4, n_z)
41

```

```

42     T_ghost_in2 = -T(:, 2);
43     T_ghost_out2 = T(:, end);
44     T_zext2 = [T_ghost_in2, T_ghost_in, T, T_ghost_out, T_ghost_out2];
45     % (n_r, n_z+4)
46     % ---- radial convective flux (1/r) d(r*u*theta)/dr
47     -----
48     u_E = u(2:end, :);
49     u_I = u(1:end-1, :);
50
51     T_r_im2 = T_ext2(1:n_r, :);
52     T_r_im1 = T_ext2(2:n_r+1, :);
53     T_r_i = T_ext2(3:n_r+2, :); % = T
54     T_r_ip1 = T_ext2(4:n_r+3, :);
55     T_r_ip2 = T_ext2(5:n_r+4, :);
56
57     T_rf_o_upwind = upwind2_face(T_r_im1, T_r_i, T_r_ip1, T_r_ip2, u_E);
58     T_rf_i_upwind = upwind2_face(T_r_im2, T_r_im1, T_r_i, T_r_ip1, u_I);
59
60     rf_o = r_f(2:end);
61     rf_i = r_f(1:end-1);
62     rc = r_c;
63
64     conv_r = (rf_o .* u_E .* T_rf_o_upwind - rf_i .* u_I .*
65     T_rf_i_upwind) ./ (rc .* dr);
66
67     % ---- axial convective flux d(w*theta)/dz
68     -----
69     w_tp = w(:, 2:end);
70     w_bt = w(:, 1:end-1);
71
72     T_z_jm2 = T_zext2(:, 1:n_z);
73     T_z_jm1 = T_zext2(:, 2:n_z+1);
74     T_z_j = T_zext2(:, 3:n_z+2); % = T
75     T_z_jp1 = T_zext2(:, 4:n_z+3);
76     T_z_jp2 = T_zext2(:, 5:n_z+4);
77
78     T_zf_p_upwind = upwind2_face(T_z_jm1, T_z_j, T_z_jp1, T_z_jp2, w_tp)
79     ;
80     T_zf_m_upwind = upwind2_face(T_z_jm2, T_z_jm1, T_z_j, T_z_jp1, w_bt)
81     ;
82
83     % At the physical inlet face the incoming transported value is the
84     % prescribed Dirichlet value theta=0, not a LUD reconstruction. The
85     % odd
86     % ghosts remain necessary for the second-order axial diffusion
87     % stencil.
88     inflow = w_bt(:, 1) > 0.0;
89     T_zf_m_upwind(inflow, 1) = 0.0;
90
91     conv_z = (w_tp .* T_zf_p_upwind - w_bt .* T_zf_m_upwind) / dz;
92
93     % ---- radial diffusion (1/r) d/dr(r*dtheta/dr)
94     -----
95     visc_r = alpha * (rf_o .* (T_ext(3:end, :) - T) ...
96     - rf_i .* (T - T_ext(1:end-2, :))) ./ (rc * dr^2);
97

```

```

90 % ---- axial diffusion d^2 theta/dz^2
91 -----
92 visc_z = alpha * (T_zext(:, 3:end) - 2.0 * T + T_zext(:, 1:end-2)) /
93 dz^2;
94 rhs = -conv_r - conv_z + visc_r + visc_z;
95 end

```

Listing 20: compute_rhs_T.m — Energy equation RHS: 2nd-order upwind convection, central diffusion

```

1 function T_new = advance_temperature(T, u, w, r_c, r_f, dr, dz, dt, Re,
   Pr)
2 % ADVANCE_TEMPERATURE Advance temperature by one explicit Euler step.
3 % Thin wrapper around compute_rhs_T; the main time loop uses
4 % compute_rhs_T directly (via ab2_step) to build the AB2 update.
5 %
6 % Returns T_new : (n_r, n_z)
7
8 rhs = compute_rhs_T(T, u, w, r_c, r_f, dr, dz, Re, Pr);
9 T_new = T + dt * rhs;
10
11 % Inlet/outlet are enforced via ghost cells inside compute_rhs_T; no
12 % direct cell-value overwrite is needed.
13 end

```

Listing 21: advance_temperature.m — Temperature time advance

B.8 Convergence Monitoring

```

1 function res = compute_residuals(u_new, u_old, w_new, w_old, T_new,
   T_old, r_c, r_f, dr, dz, dt)
2 % COMPUTE_RESIDUALS Compute convergence residuals.
3 %
4 % Returns res struct with fields:
5 % .continuity : max |div(u)|
6 % .momentum : max change rate of both u and w (interior)
7 % .temperature : max |T_new - T_old| / dt
8
9 div = compute_divergence(u_new, w_new, r_c, r_f, dr, dz);
10 res.continuity = max(abs(div(:)));
11 R_u = max(abs(u_new(2:end-1, :) - u_old(2:end-1, :)), [], 'all');
12 R_w = max(abs(w_new(:, 2:end-1) - w_old(:, 2:end-1)), [], 'all');
13 res.momentum = max(R_u, R_w) / dt;
14 res.temperature = max(abs(T_new(:) - T_old(:))) / dt;
15 end

```

Listing 22: compute_residuals.m — Residual computation

```

1 function converged = check_convergence(res, tol_cont, tol_vel, tol_temp)
2 % CHECK_CONVERGENCE Return true when all residuals are below tolerances
3 .
4
5 converged = (res.continuity < tol_cont) && ...
6             (res.momentum < tol_vel) && ...
7             (res.temperature < tol_temp);
8 end

```

B.9 Analytical Reference Solutions

```

1 function w_an = analytical_velocity(r_star)
2 % ANALYTICAL_VELOCITY Hagen-Poiseuille fully developed velocity profile
3 %
4 %  $w^* = 2 * (1 - (2*r^*)^2)$ 
5
6     w_an = 2.0 * (1.0 - (2.0 * r_star).^2);
7 end

```

Listing 24: analytical_velocity.m — Hagen-Poiseuille axial velocity profile

```

1 function theta_an = analytical_temperature(r_star, z_star, Re, Pr)
2 % ANALYTICAL_TEMPERATURE Fully developed temperature profile (approved
3 % form).
4 %
5 %  $\theta = 4*z + Re*Pr * [1/2*(2r)^2 - 1/8*(2r)^4 - 7/48]$ 
6 %
7 % Obtained by direct integration of the non-dimensional energy
8 % equation with
9 % the wall condition  $d\theta/dr = Re*Pr$ ; the constant  $-7/48$  follows
10 % from the
11 % exact global energy balance ( $\theta_{bulk} = 4*z$  for all  $z \geq 0$ ).
12 % Gives
13 %  $Nu = 48/11$ . Full derivation and sources: Report/
14 % nusselt_number_analysis.md.
15 % (An earlier transcription of the assignment omitted the  $1/2$  factor
16 % on the
17 % radial terms and used  $-3/4$ ; that form is not used anywhere in this
18 % codebase.)
19
20     eta = 2.0 * r_star;
21     theta_an = 4.0 * z_star + Re * Pr * (0.5 * eta.^2 - 0.125 * eta.^4 -
22     7.0/48.0);
23 end

```

Listing 25: analytical_temperature.m — Fully developed temperature profile ($Nu = 48/11$)

B.10 Plotting Helper

```

1 function cmap = diverging_bwr(n)
2 % DIVERGING_BWR Blue-white-red diverging colormap (no toolbox
3 % dependency).
4 %
5 %  $cmap = diverging\_bwr(n)$  returns an  $n$ -by-3 colormap running from blue
6 % (low) through white (mid) to red (high), suitable for signed fields
7 % (e.g. radial velocity) plotted with a symmetric color axis centred
8 % on
9 % zero via  $caxis([-lim, lim])$ .
10
11     if nargin < 1
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

```

10     n = 256;
11     end
12
13     blue = [0.13, 0.40, 0.85];
14     white = [1.00, 1.00, 1.00];
15     red = [0.80, 0.10, 0.10];
16
17     half = floor(n / 2);
18     lower = [linspace(blue(1), white(1), half)', ...
19             linspace(blue(2), white(2), half)', ...
20             linspace(blue(3), white(3), half)'];
21     upper = [linspace(white(1), red(1), n - half)', ...
22             linspace(white(2), red(2), n - half)', ...
23             linspace(white(3), red(3), n - half)'];
24     cmap = [lower; upper];
25 end

```

Listing 26: diverging_bwr.m — Blue–white–red diverging colormap for contour plots

References

- [1] R. K. Shah and A. L. London, *Laminar Flow Forced Convection in Ducts*, Supplement 1 to *Advances in Heat Transfer*, Academic Press, New York, 1978. (Circular duct, H1 boundary condition: $Nu = 48/11$.)
- [2] F. P. Incropera and D. P. DeWitt, *Fundamentals of Heat and Mass Transfer*, Wiley, New York. (Internal flow chapter: fully developed laminar flow in a circular tube with uniform surface heat flux, $Nu_D = 48/11 \approx 4.36$.)
- [3] W. M. Kays, M. E. Crawford, and B. Weigand, *Convective Heat and Mass Transfer*, McGraw-Hill, New York. (Fully developed laminar circular-tube flow, constant heat rate per unit length: $Nu = 48/11$.)
- [4] R. B. Bird, W. E. Stewart, and E. N. Lightfoot, *Transport Phenomena*, Wiley, New York. (The same fully developed constant-wall-flux tube problem is worked out and yields $Nu = 48/11$.)